Lecture Note 7: Memory-Management Strategies

CSS225 Operating Systems

School of ICT, SIIT, Thammasat University

Scenario: Running an Application Program

1. An application program in the hard disk.

→

2. When we run this program, the program is converted into the corresponding machine code (= a process).

→

3. This machine code will be store in the main memory. (We basically referred it as a **process**)

→

4. A CPU executes this process.

What we will learn!

We will study 3 basic strategies/methods on storing processes into the main memory.

Each method will:

1) Find a free space in the main memory to store a process.
2) Map logical addresses into physical addresses

Outlines

1. Background
2. Swapping
3. Method 1: Contiguous Memory Allocation
4. Method 2: Segmentation
5. Method 3: Paging
6. Address Conversion in Paging
7. Internal Fragmentation in Paging
8. Page Table and Frame Table

1. Background

Basic Infor: Roles of the Main Memory

1) The main memory consists of a large array of addressable bytes (or words, depending on the architecture), each identified by a unique memory address. It serves as the primary storage for active processes. In conclusion:
   a) The main memory is divided into rows.
   b) Each row stores a binary number (called a word = instructions/data).
   c) Each row has an address (called a physical address).

2) The main memory is typically divided into two regions:
   a) kernel space (reserved for the operating system and privileged processes)
   b) user space (allocated for user applications and processes).

Basic Infor: Roles of the Main Memory

3) The CPU can directly access two types of general-purpose storage:

- main memory and registers.

Registers provide the fastest access speeds but have limited capacity, while main memory offers larger storage but slower.

4) The CPU can only execute

- instructions and manipulate data stored in main memory or registers.

If the required data is not in memory (e.g., it resides in secondary storage), it must be loaded into RAM via memory management techniques such as demand paging before the CPU can process it. .

Basic Infor: Memory Terminology

A memory unit (in the main memory) stores binary information, including data and instructions, in groups of bits known as words.

1) **Memory Word**:

   a) A word is a fixed-sized group of bits (memory cells) used to represent instructions, numerical values, or other types of data.

   b) The size of a word, known as the word length, varies depending on the system architecture. Modern computers typically use word lengths ranging from 8 to 64 bits, but some high-performance or specialized systems may use larger word sizes (e.g., 128 bits). The word size directly affects the system's processing power, memory addressing, and data transfer efficiency.

Basic Infor: Memory Terminology

2) **Byte**: A byte is a standard unit of data storage, always consisting of eight bits. Since a word is composed of multiple bits, word sizes can also be expressed in bytes. For example:
   - 8-bit word = 1 byte
   - 16-bit word = 2 bytes
   - 32-bit word = 4 bytes
   - 64-bit word = 8 bytes
3) **Word-addressable memory**:
   - In this lecture, we assume that the word-addressable memory, where each address corresponds to an entire word rather than a byte.
   - Remark: In modern computing, byte-addressable memory is commonly used, meaning each memory address refers to a single byte rather than a whole word. This allows flexible access to smaller data units within a word.

Basic Infor: Memory Addresses

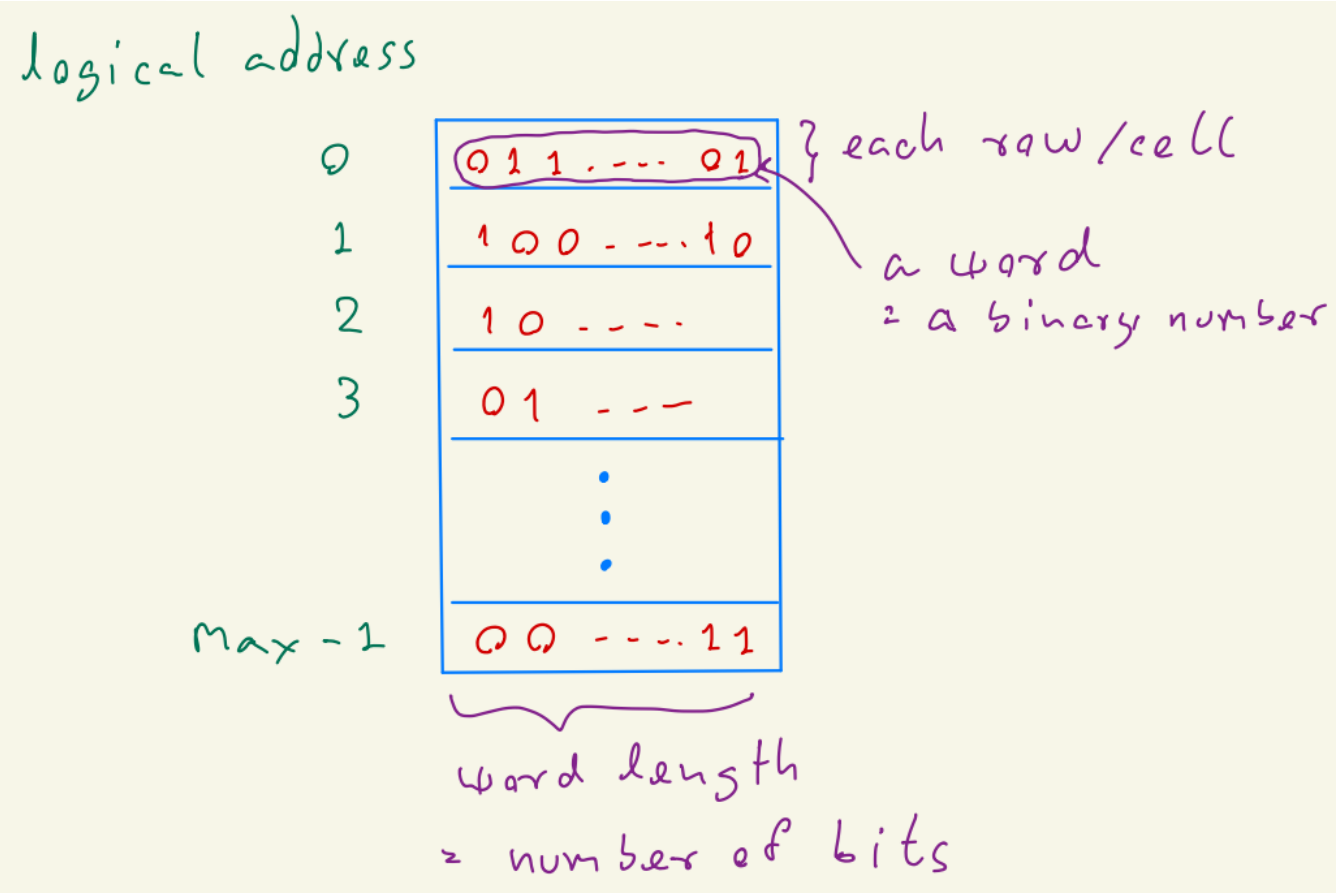
A memory address is a unique identifier assigned to a specific word or byte in memory, allowing data to be retrieved or stored at that location.

1) Each word or byte in memory has a unique address, which serves as an index for accessing data efficiently. The addressing scheme depends on whether the system is byte-addressable (where each address refers to a byte) or word-addressable (where each address refers to a word).
   - In this lecture note, we use word-addressable. (Remark: Many modern architectures use byte-addressable memory for greater flexibility in data access).
2) Memory addresses are typically represented using various number systems:
   - **Binary** (e.g., 1010100110_2) – used internally by the computer.
   - **Octal** (e.g., 523_8) – used in older systems but now less common.
   - **Hexadecimal** (e.g., 0x1A3F) – widely used in modern computing because it provides a compact representation of binary addresses.
   - **Decimal** (e.g., 6723_{10}) – often used in documentation for readability.
3) When referencing a specific memory location, its address is used to locate and access the stored data or instruction.

Logical Diagram of a Process

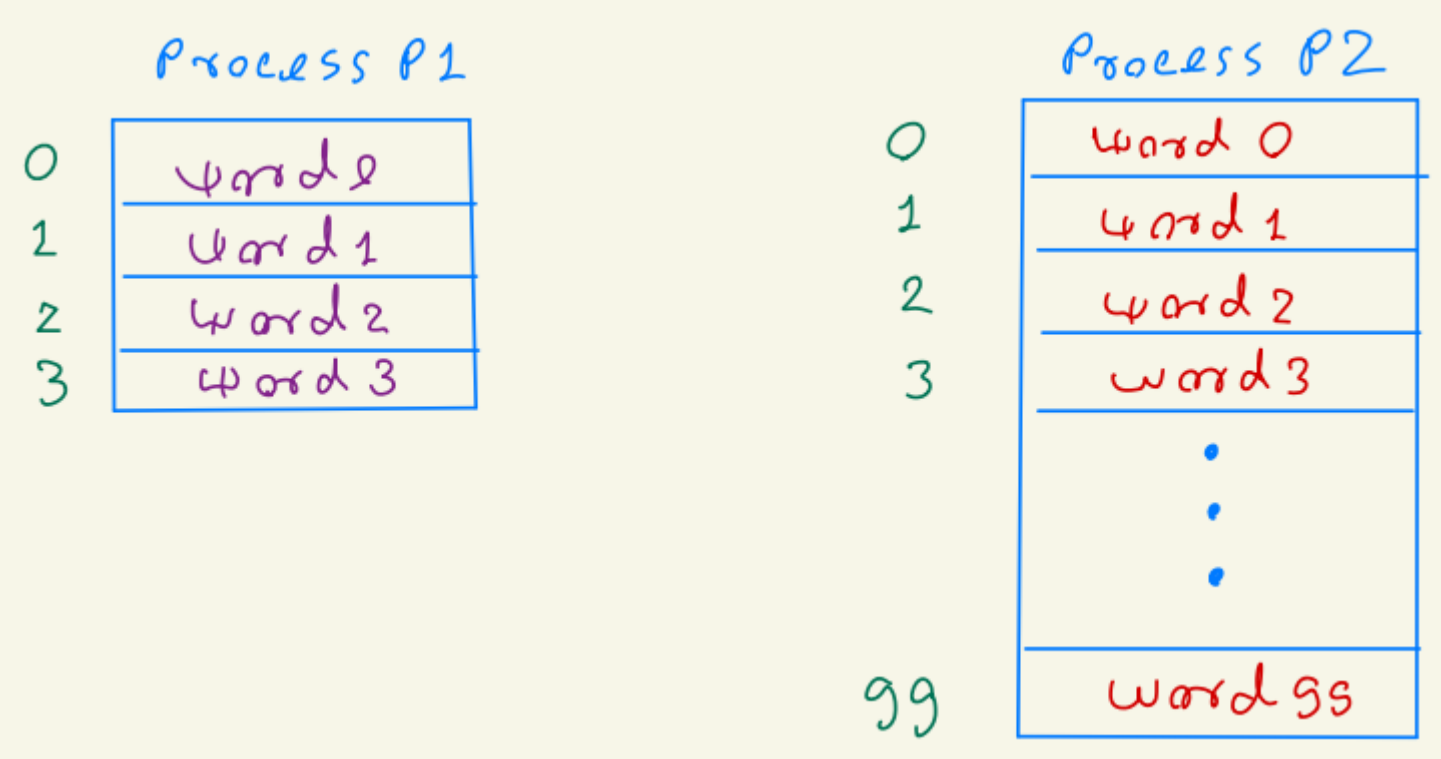
The process structure assumed in this lecture note is shown below.

1) A process consists of Max rows.
2) Each row (a cell) has a logical address (an integer between 0 and Max-1).
3) Each row contains a set of machine code (a binary number) called a word (word-addressable).
4) The size of a word is measured as a number of bits, for example, 8 bits, 16 bits, etc.



Example: Logical Diagram of a Process

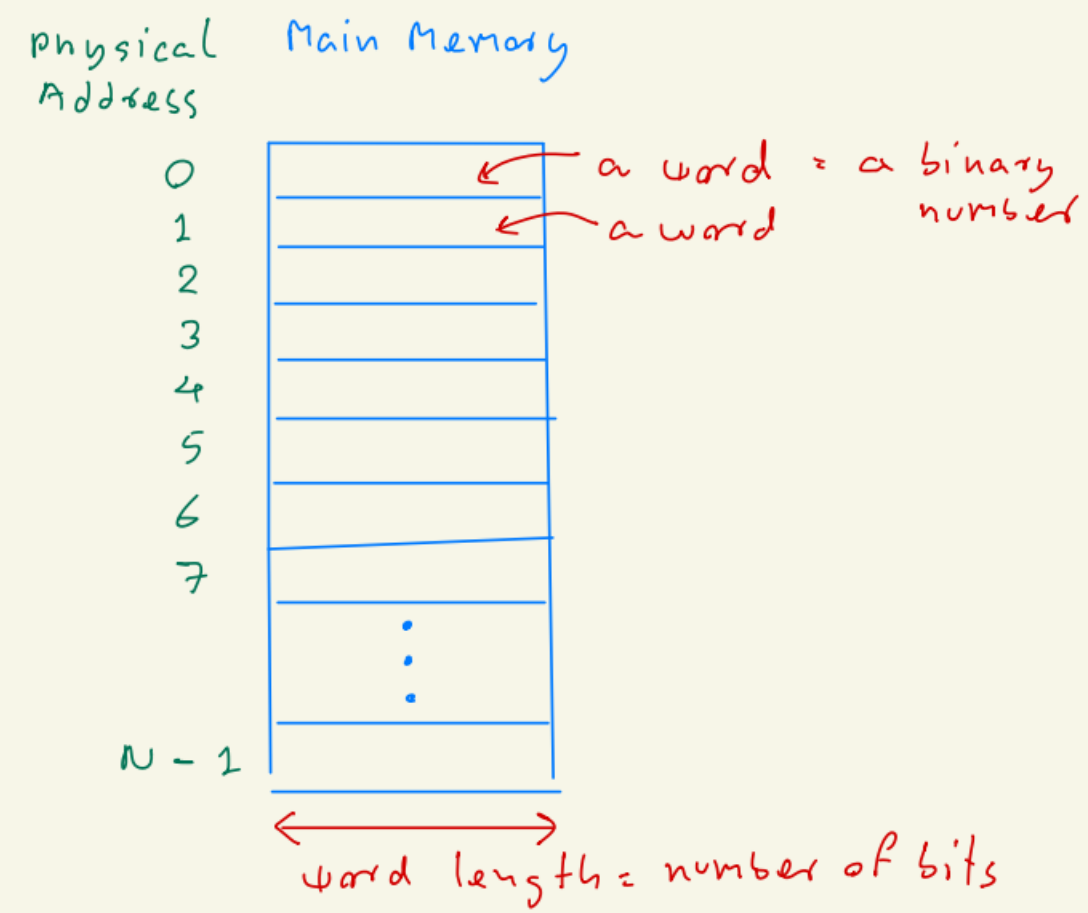
- Process P1 consists of 4 words and has the logical address: 0, 1, 2, 3.
- Process P2 consists of 99 words and has the logical address: 0, 1, 2, 3, …, 99.



Main-Memory Diagram Storing Processes

The main-memory structure assumed in this lecture note is shown below.

1) The main memory is divided into the kernel space and user space.
2) User processes are stored in the user space.
3) The main memory consists of N rows.
4) Each row has a physical address (an integer between 0 and N-1).
5) Each row can store 1 word from a process.
6) The size of words in a process is the same with that in the main memory.



Example 1: Size of a process

Consider a process consisting of 256 words. Each word contains 16 bits.

1) How many the logical addresses do we have in this process? What are them?

2) What is the size of this process?
   a) Unit of bits:
   b) Unit of bytes (B):
   c) Unit of kilobytes (K bytes or KB):

Example 2: Size of a process

Consider a process with the size of 1K words of 16 bits each. What is the size of this process?

Ans:

- Since 1K = 2^{10} = 1024 and 16 bits (=1 word) constitute two bytes, we can say that the process size is 2048 = 2K bytes.
- The figure in the next page shows possible contents of the first three and the last three words of this process. Each word contains 16 bits that can be divided into two bytes. The words are recognized by their decimal address from 0 to 1023. The equivalent binary address consists of 10 bits. The first address is specified with ten 0's; the last address is specified with ten 1's, because 1023 in binary is equal to 1111111111.

Example 2: Size of a process

Contents a process of a 1024 x 16 bits.

Memory address		Memory content
Binary	Decimal	
0000000000	0	1011010101011101
0000000001	1	1010101110001001
0000000010	2	0000110101000110
	⋮	⋮
1111111101	1021	1001110100010100
1111111110	1022	0000110100011110
1111111111	1023	1101111000100101

Logical Address vs Physical Address

1) A logical address (also known as a virtual address) is an address generated by the CPU when a program accesses memory.
   - This address is process-specific and exists within the logical address space of a running program.
   - The logical address is used by the program.
2) A physical address is the actual location in main memory (RAM) where the data or instruction is **actually** stored.
3) The logical address space consists of all possible logical addresses that a process can generate during its execution.
4) The physical address space consists of all physical memory locations/addresses.

Mapping from Logical to Physical addresses

The computer system will store a process in an area(s) in the main memory. Knowing which physical addresses actually storing this process is necessary.

- The mapping from logical to physical addresses is handled by a specialized hardware component called the Memory Management Unit (MMU).
- The MMU translates logical addresses to physical addresses dynamically at runtime

Memory Management

Memory management is a core function of the operating system that manages main memory efficiently. It ensures that processes are allocated sufficient memory, tracks memory usage, and moves processes between main memory and secondary storage (disk) when necessary.

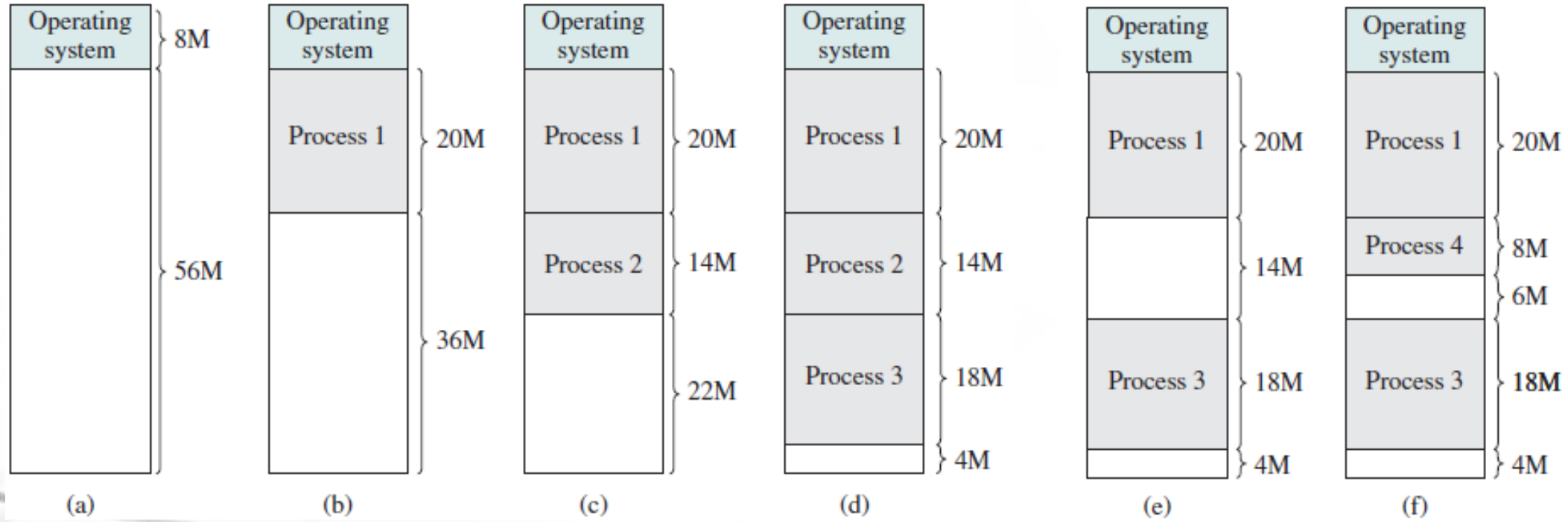
Three Main Tasks of Memory Management

1) **Memory Allocation**
   - The OS finds and allocates free memory spaces in main memory for processes.
   - It must ensure efficient use of available memory by employing allocation techniques such as contiguous allocation, segmentation, or paging.
2) **Dynamic Memory Management**
   - The OS dynamically assigns memory to processes as needed and frees memory when processes terminate or release unused memory.
   - This task involves managing memory fragmentation, which can be internal (wasted space within allocated blocks) or external (free space scattered across memory).
   - Common allocation strategies include first-fit, best-fit, and worst-fit.
3) **Address Translation (Logical to Physical Address Mapping)**
   - The OS and Memory Management Unit (MMU) handle logical-to-physical address translation so that processes can use logical addresses independently of actual physical memory locations.
   - This is crucial in virtual memory systems, where processes can access more memory than physically available through techniques such as demand paging.

Example: Memory Management

Examples on:

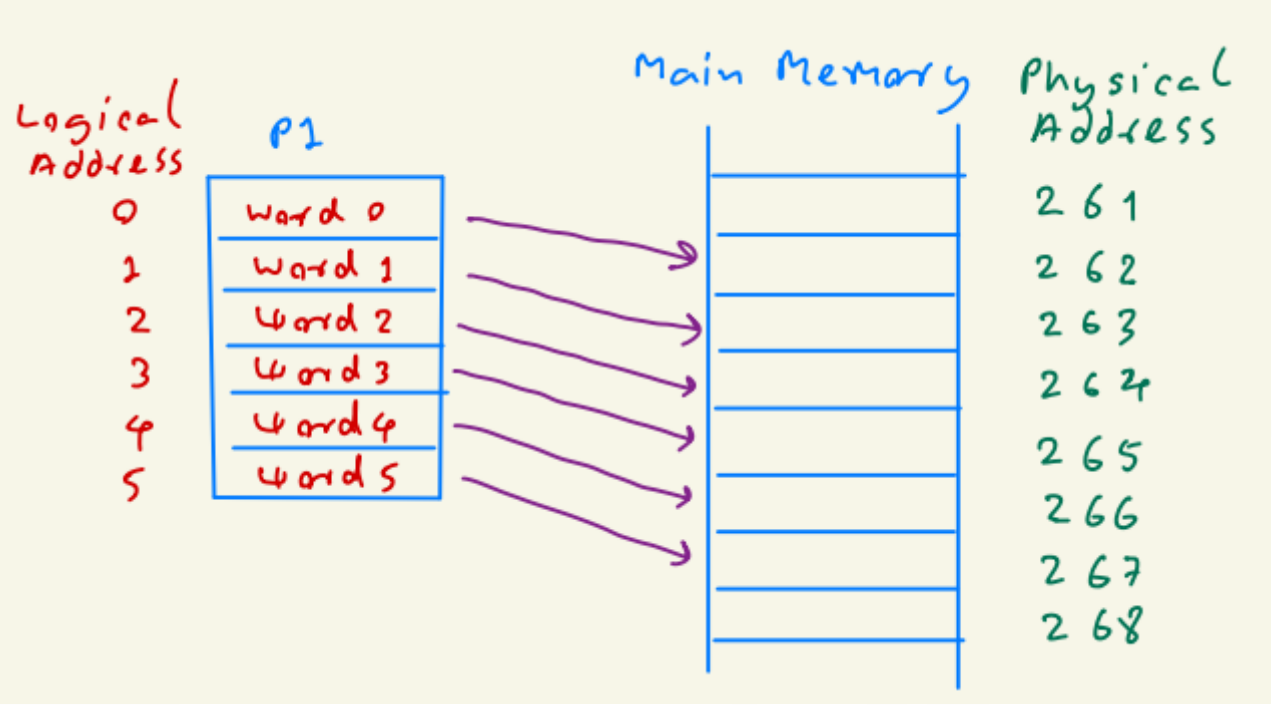
- Task 1: Find a free space in the main memory to store processes.
- Task 2: Dynamically allocate portions of memory to processes at their request, and free it for reuse when no longer needed.



Example: Memory Management

Examples on:

- Task 3: Map process's logical addresses to physical addresses.



Big Picture

a) Processes will be stored in different areas in the main memory.

b) We can identify an instruction inside a process by specifying its logical address or the corresponding physical address.

2. Swapping

Swapping: Definition

Swapping is a memory management technique in which a process in the waiting state (i.e., not actively using the CPU) is temporarily moved from main memory (RAM) to secondary storage (backing store, e.g., a hard drive or SSD). The process can be swapped back into memory when needed, allowing the system to manage multiple processes efficiently.

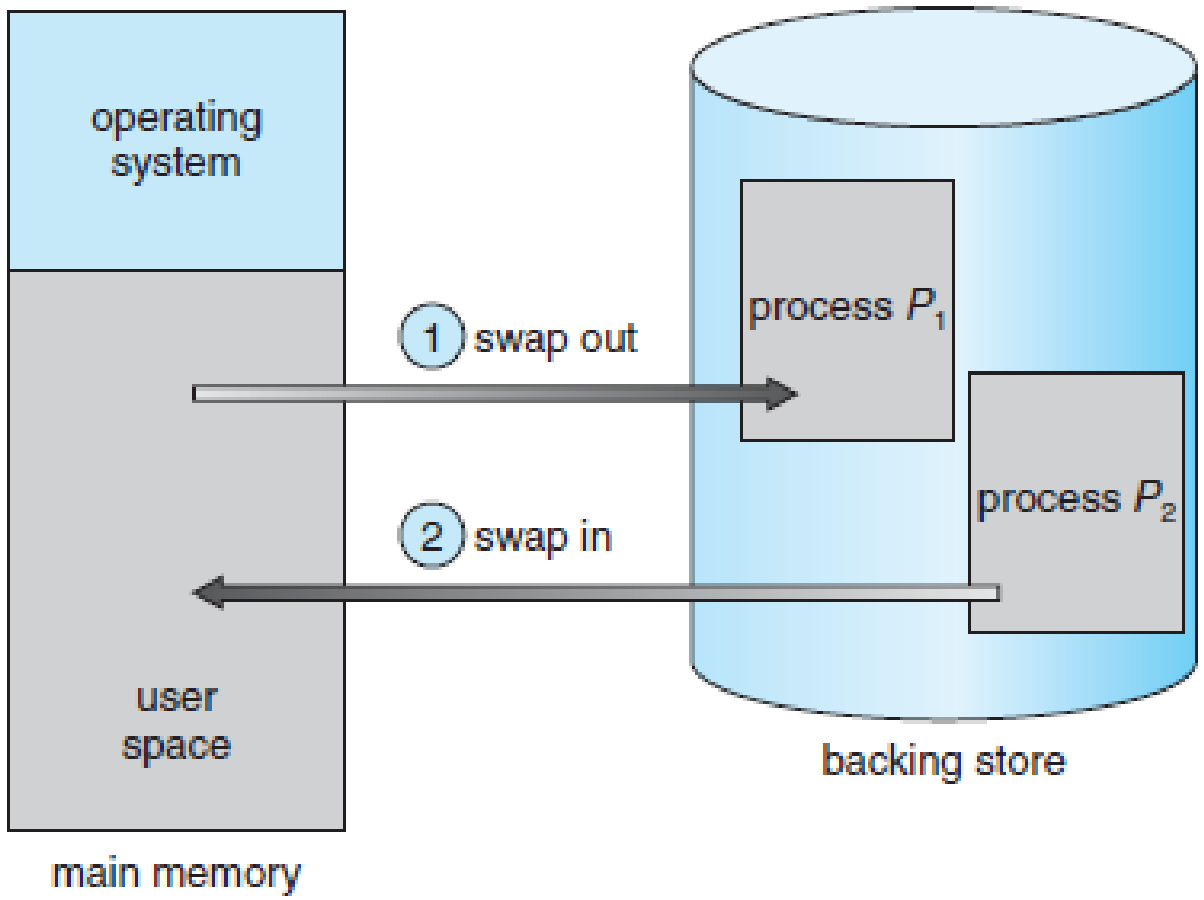
Why do we need swapping?

1) The OS moves an idle or low-priority process from the main memory to a swap space on disk to free up memory for active processes. When the process is required again, it is reloaded into the main memory and resumes execution from where it left off.
2) Swapping increases the number of processes that can run concurrently by allowing the OS to manage more processes than the available physical memory can accommodate.
   - However, frequent swapping can lead to performance degradation, known as thrashing, where excessive disk I/O slows down system performance.

Standard Swapping

Standard swapping involves moving a whole process between the main memory and a backing store (e.x., hard drive).

- Swapping in → A process stored in the backing store is moved to the memory (to be executed).
- Swapping out → A process stored in the memory is moved to the backing store (to provide a space for another process).



Main Disadvantage

The swap time of a process is the time duration needed to load the process from the backing store to the memory (or vice versa).

Example: Consider a process with the size 100 MB stored in the hard drive with a transfer rate 50 MB/s. The swap is 100/50 = 2 seconds (it is a lot as compared a CPU execution time in unit of milliseconds or microseconds).

Swapping vs. Paging

Standard swapping is not currently used in modern operating systems. Since swapping moves entire processes in and out of memory, it requires too much swapping time and provides too little execution time to be a reasonable memory-management solution.

- Modern systems often use paging-based virtual memory instead of full-process swapping to improve efficiency.
- Paging (in Lecture Note 8) moves fixed-size memory blocks (pages) between the main memory and disk.

Relocation: A Fundamental Aspect of Process Management in Main Memory

1) **Shared Memory in Multiprogramming Systems**: In a multiprogramming environment, the available main memory is shared among several processes concurrently. Since the operating system dynamically loads and unloads processes, a programmer typically cannot predict which other processes will be residing in memory when their program is executed. To address this, processes include relocation information so that the operating system can adjust memory references as needed.
2) **Dynamic Process Placement and Swapping**: To maximize processor utilization, the operating system maintains a large pool of ready processes. When a process is not immediately required (e.x., a waiting process), it may be swapped out to a secondary storage (such as a hard disk) to free up main memory for other active processes. When this process is swapped back in, it might be loaded into a different memory region. Dynamic relocation during this swap-in phase is facilitated by hardware components like the Memory Management Unit (MMU), which adjust the process's address references based on its new location.

Relocation: A Fundamental Aspect of Process Management in Main Memory

3) **Multiple Relocation Events**: During its lifetime, a process might be relocated multiple times. Each relocation involves updating the mapping between the process's logical address space and its physical location in memory. This mapping is maintained by data structures such as page tables or segment tables, which allow the MMU to translate logical addresses to the correct physical addresses regardless of where the process is currently loaded.
4) **Implications for Memory Addressing:** Because a process's physical memory location is not fixed, the system must support dynamic address translation. The logical address space remains constant for the process, while the operating system, with support from the MMU, maps these logical addresses to physical locations in main memory. This decoupling of logical and physical addresses enables efficient memory management and flexibility in process scheduling and swapping.

3. Method 1: Contiguous Memory Allocation

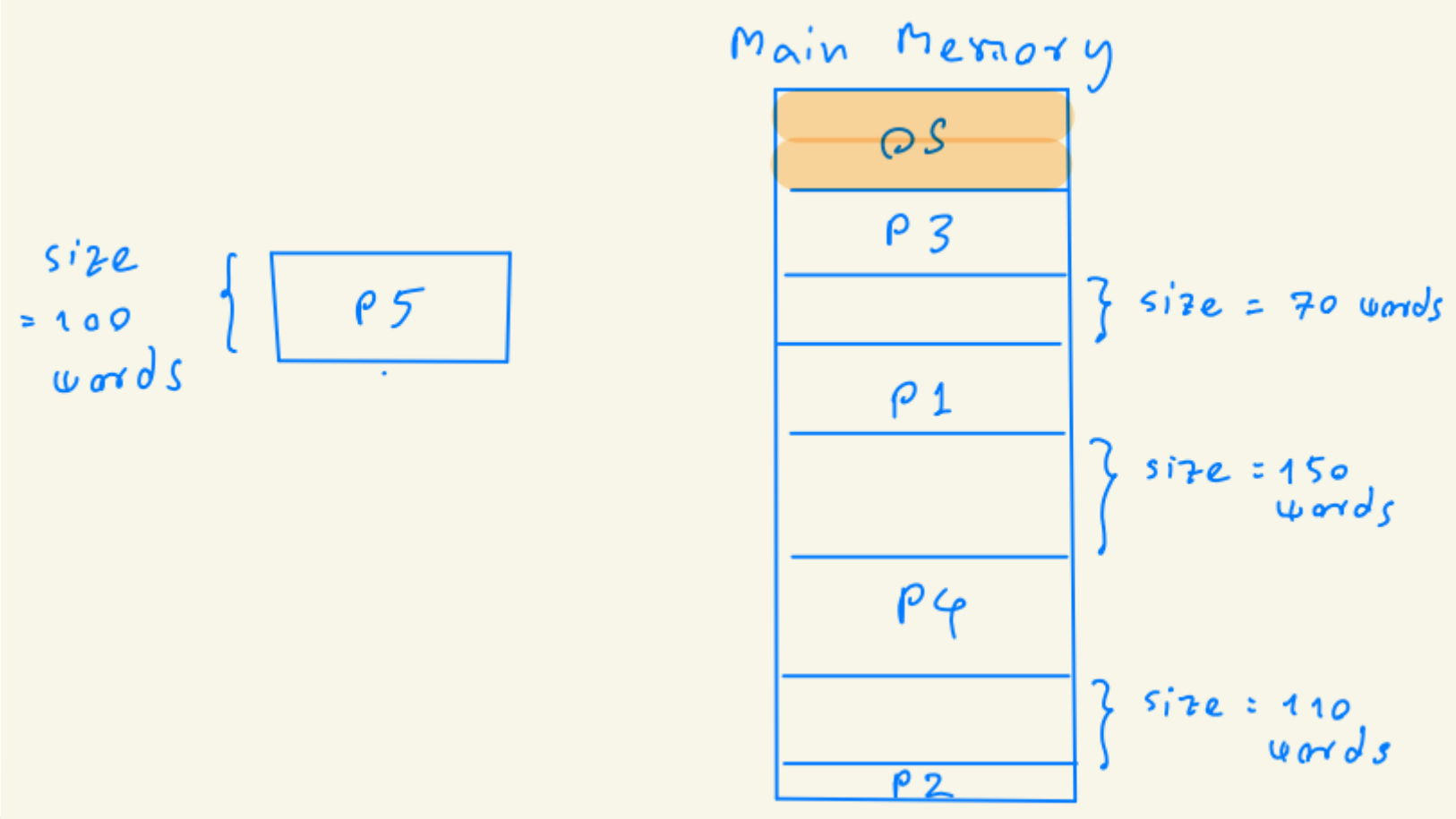
Contiguous Memory Allocation

Contiguous memory allocation is a memory management technique where each process is assigned a single contiguous block of memory.

- The process cannot be split across multiple memory locations, meaning the operating system (OS) must find a single free memory region large enough to accommodate/store the entire process.
- We will learn:
  a) Strategy to select free spaces in the main memory to store processes.
  b) Logical-physical address translation (mapping).

Example: Contiguous Memory Allocation

The following example shows how to store processes in the main memory by using the contiguous memory allocation.



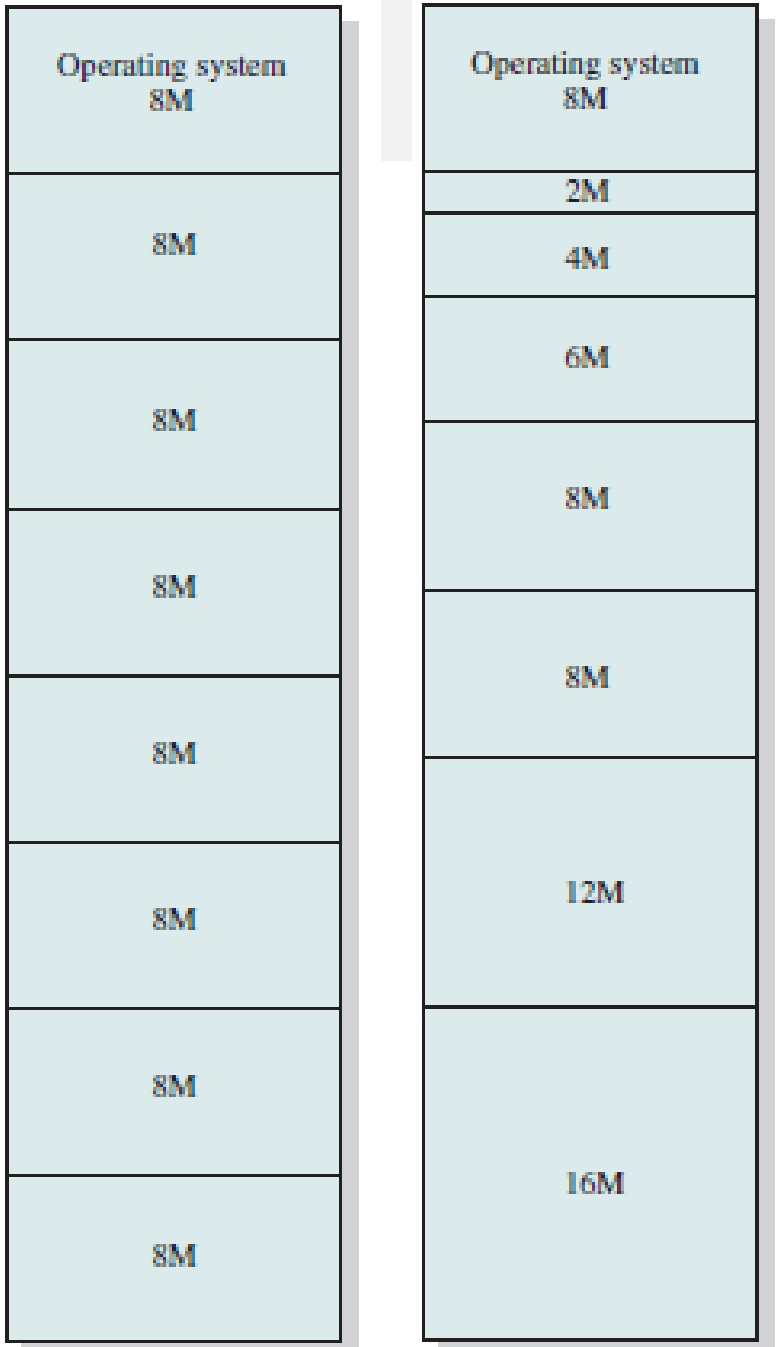
Methods

Types of contiguous memory allocation methods:

1) Fixed-size partition method
   a) Equal-size partition
   b) Unequal-size partition
2) Variation partition method

Method 1: Fixed-Sized Partition

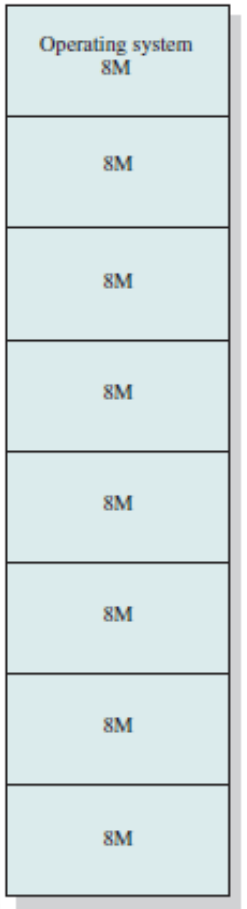
- Fixed-sized partitions:
  - The memory is divided into several fixed-sized partitions.
  - Each partition contain exactly one process.
  - The degree of multiprogramming is bounded by the number of partitions.
- There are two types of fixed-sized partitions:
  a) Equal-size partition
  b) Unequal-size partition



(a) Equal-size partitions (b) Unequal-size partitions

Example: Equal-size partition

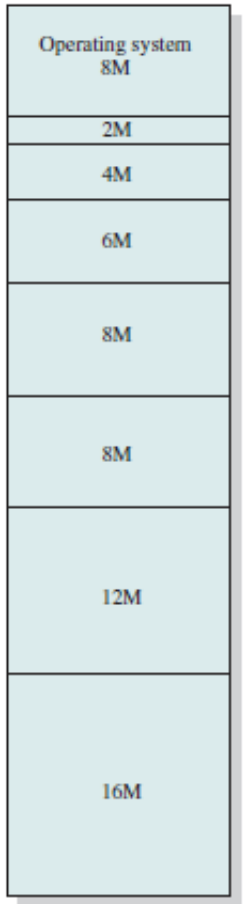
Based on the equal-size partition, the main memory's area is divided into the following partitions/spaces. Assume that we have 3 processes: P1 (8 MB), P2 (5 MB), P3 (16 MB). Which partitions will store these processes?



(a) Equal-size partitions

Example: Unequal-size partition

Based on the unequal-size partition, the main memory's area is divided into the following partitions/spaces. Assume that we have 3 processes: P1 (8 MB), P2 (5 MB), P3 (16 MB). Which partitions will store these processes?



(b) Unequal-size partitions

A Problem from using the Fixed-Size Partition: Internal Fragmentation

Internal fragmentation occurs when an allocated memory block (partition) is larger than the actual size needed by the process.

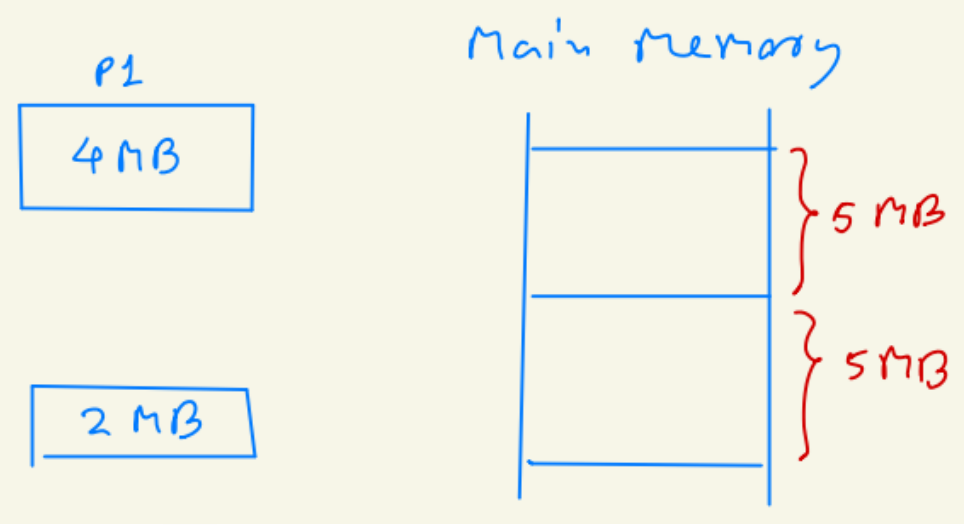
- As a result, there will be unused space left in this partition. This unused space is called internal fragmentation.
- The unused space within the allocated memory block is considered wasted or fragmented since no other processes can be stored in this unused space.

Example: Internal Fragmentation

Consider an equal-size partition with the partition size equal to 5 MB. Answer the following questions.

a) What is the size of the internal fragmentation when we store the process with the size of 4 MB in a partition?

b) What is the size of the internal fragmentation when we store the process with the size of 2 MB in a partition?



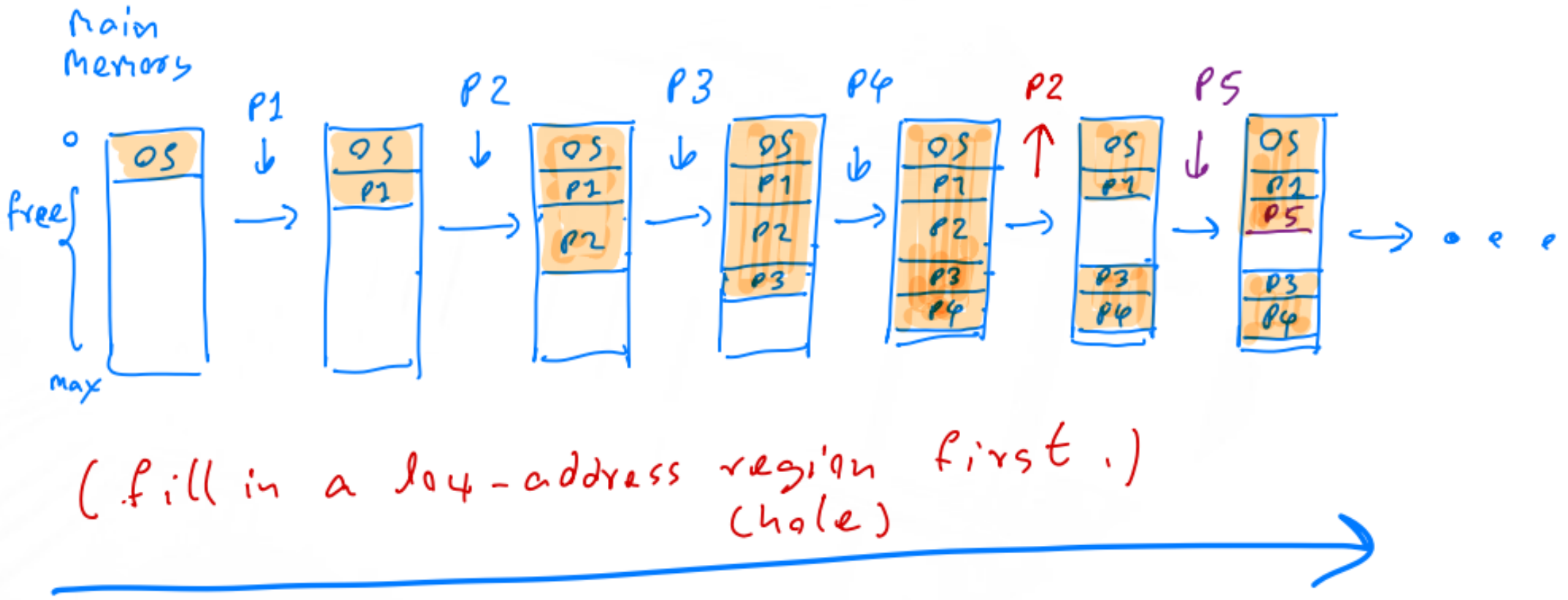
Method 2: Variation Partition

In the variation-partition method, the space in the main memory is not pre-divided into fixed-size partitions, as opposed to methods like fixed-size partitioning. Instead, memory is dynamically allocated based on the process's size and available free space. As a result:

1) **Dynamic Allocation of Memory**:
   - A process is placed into a free region of memory, called a hole, which is large enough to accommodate the process's size.
   - These holes are regions of free memory that vary in size and location. As processes enter and exit the system, the number and size of these holes will change.
   - When a process is loaded, the operating system searches for a suitable hole in memory and assigns the process to that region. The allocation strategy may involve methods like first-fit, best-fit, or worst-fit.
2) **Variable-Sized Holes**:
   - Unlike fixed-size partitioning, where each process is assigned to a fixed-size partition, the variation-partition method allows processes to fit into memory holes of varying sizes.
   - As result, memory utilization can be more efficient compared to fixed-size partitions, particularly for processes of variable sizes.

Example: Variation Partition

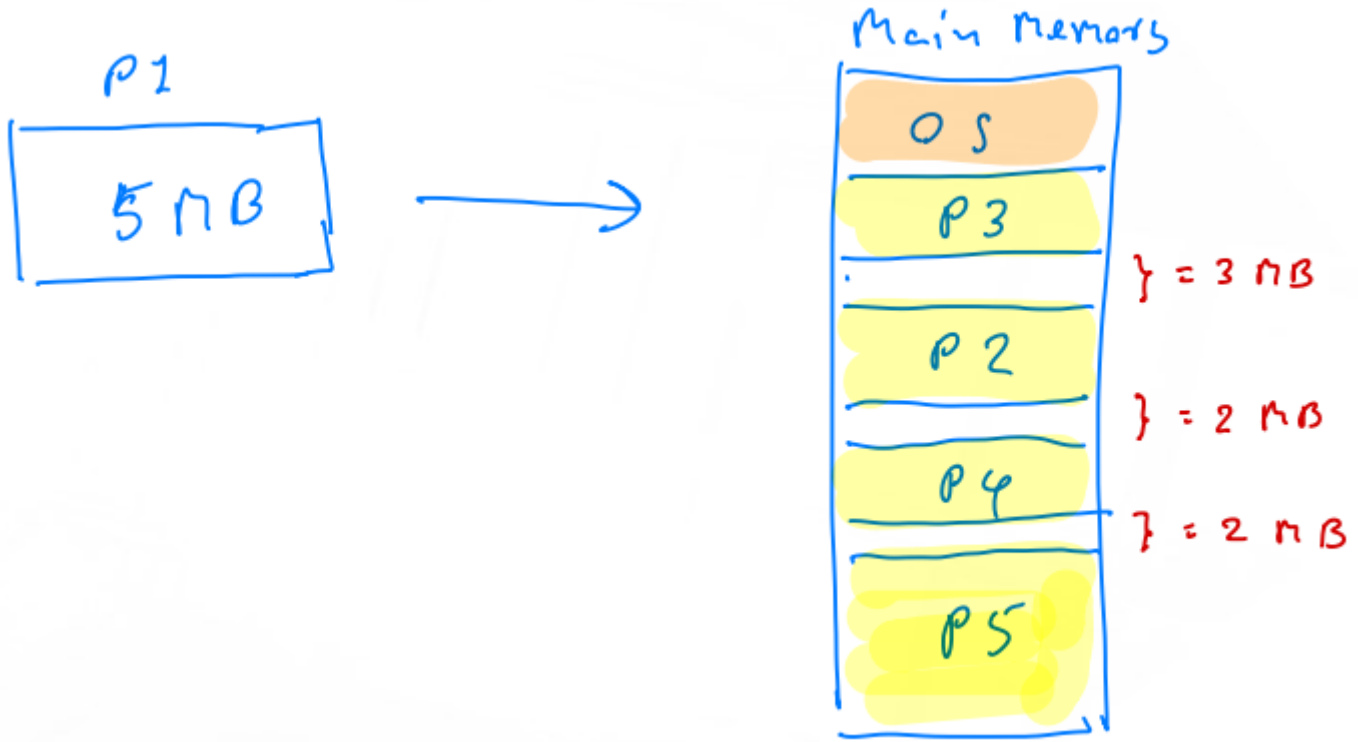
To understand the concept of the variation partition, the following example shows how to store processes in the main memory by using the variation partition method.



A Problem from using the Variation Partition: External Fragmentation

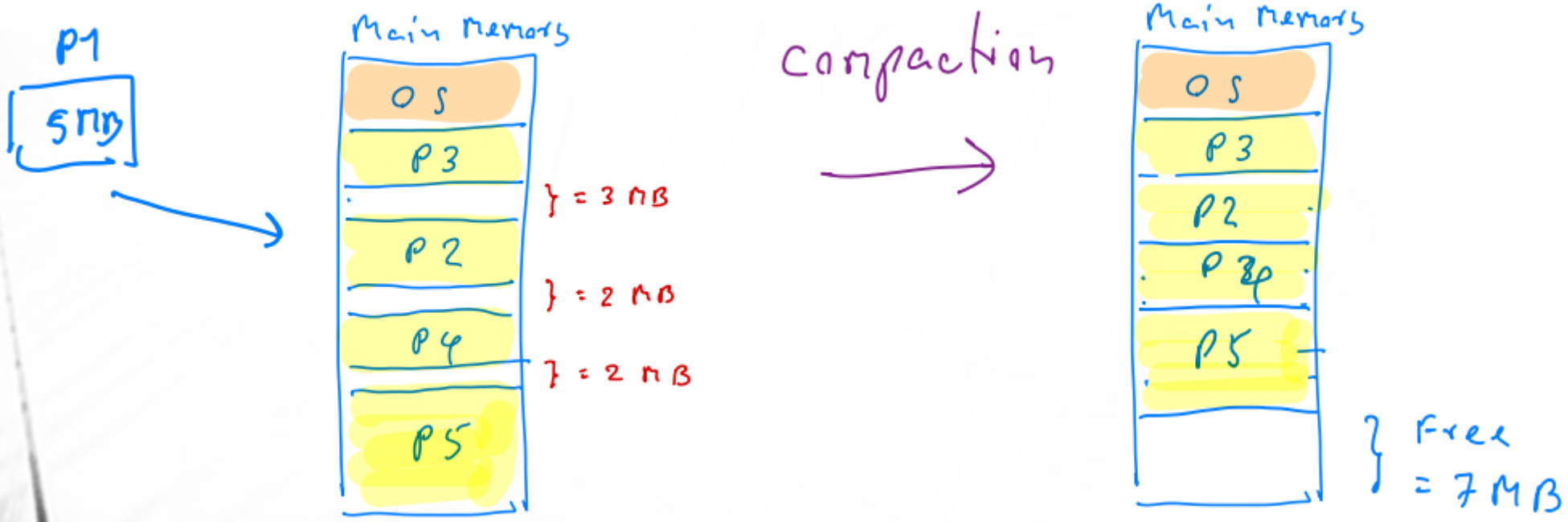
Over time, as processes are allocated and deallocated, the system can experience external fragmentation, where free memory is split into small, non-contiguous regions.

- This leads to a situation where no single hole is large enough to accommodate a new process, despite having enough total free memory.



External Fragmentation: Solutions

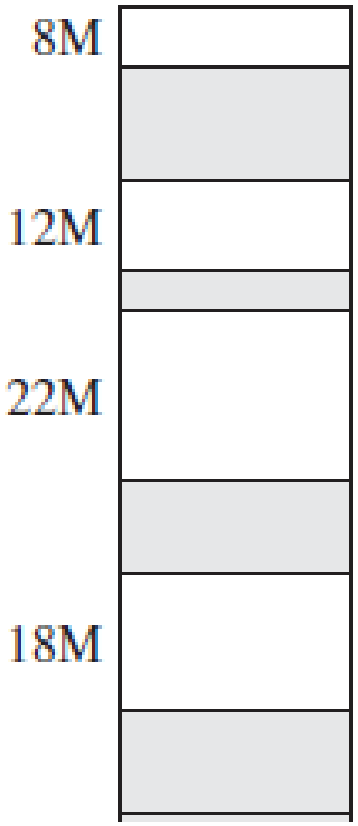
1) Compaction: Rearranges memory contents to consolidate all free space into a single large block, reducing fragmentation.
2) Segmentation and Paging: We allow the logical address space of the process to be noncontiguous and, thus, allow the process to store wherever the memory is available (in noncontiguous way).



Dynamic Storage-Allocation Strategy

A. The dynamic storage-allocation strategy is applied to unequal-size partition and variation partition.

B. (Example on the next page.) Consider a scenario that we are storing a process (16 MB) into the main memory. Assume that there are several possible free space regions (whose sizes are larger than the size of this process) available in the main memory. Which region should we select to store this process?



C. Common strategies are

1) First fit. Allocate the first free space that is big enough (searching from low address towards high address).

2) Best fit. Allocate the smallest free space that is big enough. We must search for the entire list.

3) Worst fit. Allocate the largest free space. We must search for the entire list.

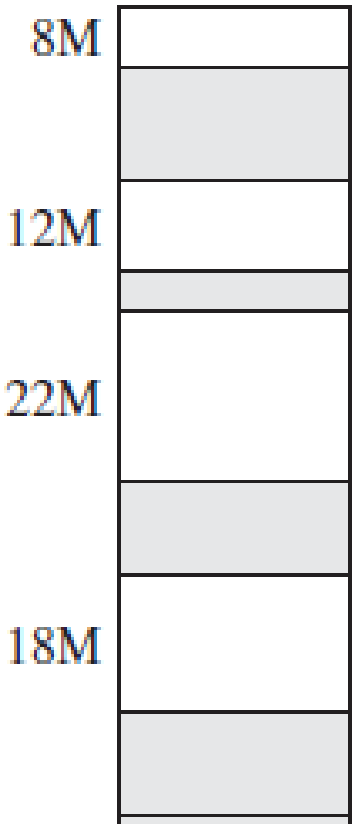
Example: Dynamic Storage-Allocation Strategy

Consider the allocation of a process of size 16 MB to a memory whose free space is shown aside. Which free regions should we store this process?

1) Using the first fit.

2) Using the best fit.

3) Using the worst fit.



Logical-Address and Physical-Address Mapping

In the contiguous memory allocation, we are storing a process into the main memory with the following strategy:

"storing the whole process in a free space region in the main memory."

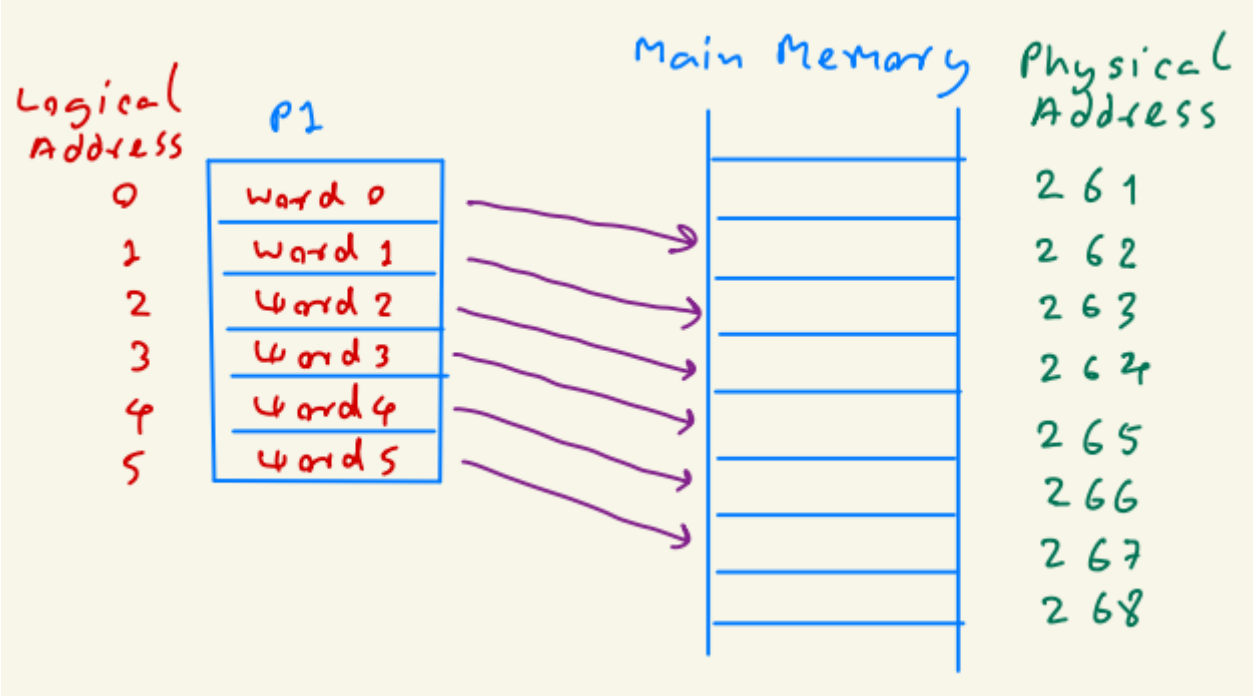
We can find the corresponding physical address of the words in this process as follows.

1) A process contains the number of words equal to "limit". The logical address starts from 0 to limit-1.
2) The first word of this process (i.e., the word at the logical address is 0) is stored at the physical address "base" of the main memory.
3) The corresponding physical address to store the word of the logical address k of the process can be computed from:

Physical Address = base + k

Example: An Illustration

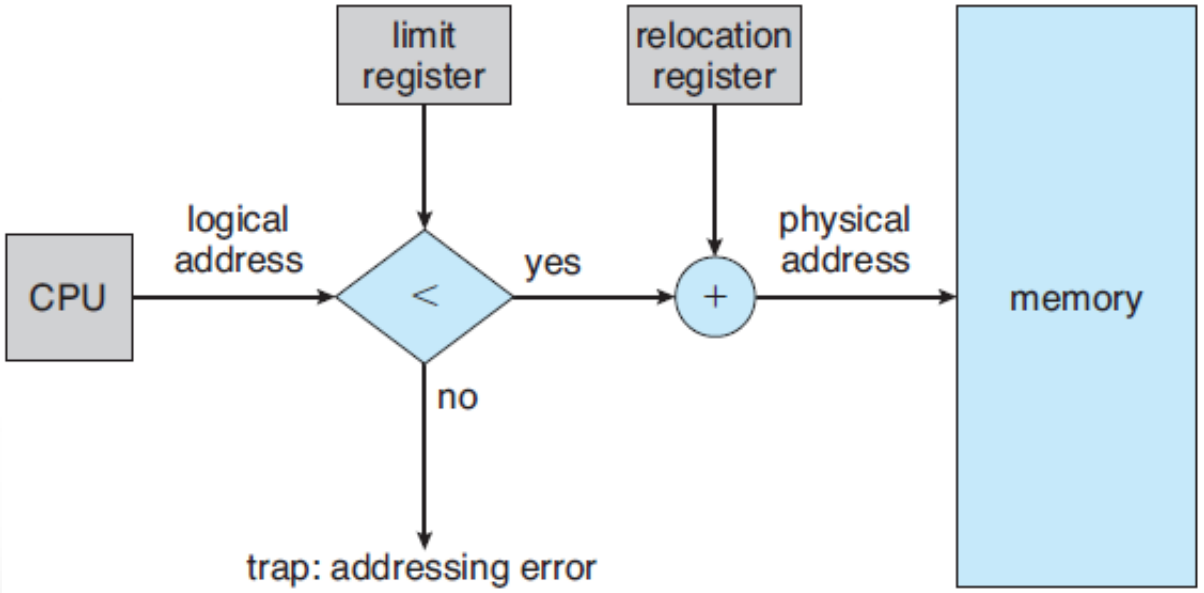
Storing the process P1 in the main memory by using the contiguous memory allocation.



Implementation of Logical-Physical-Address Mapping

To find "physical address" of "logical address", we have the following steps.

1) Compare the "logical address" to the "limit" value in the limit register.
   - If "logical address" > or = "limit": error.
   - If "logical address" < "limit": go to Step 2.
2) By adding "logical address" with "base" value stored in the relocation register, we obtain the corresponding "physical address".



Example: Logical-Address and Physical-Address Mapping

Consider a process containing 100 words, stored in free space within the main memory, starting at the physical address 3155.

1) What is the physical address corresponding to the logical address 30?

2) What is the physical address corresponding to the logical address 99?

3) What is the physical address corresponding to the logical address 100?

4) What is the physical address corresponding to the logical address 150?

4. Method 2: Segmentation

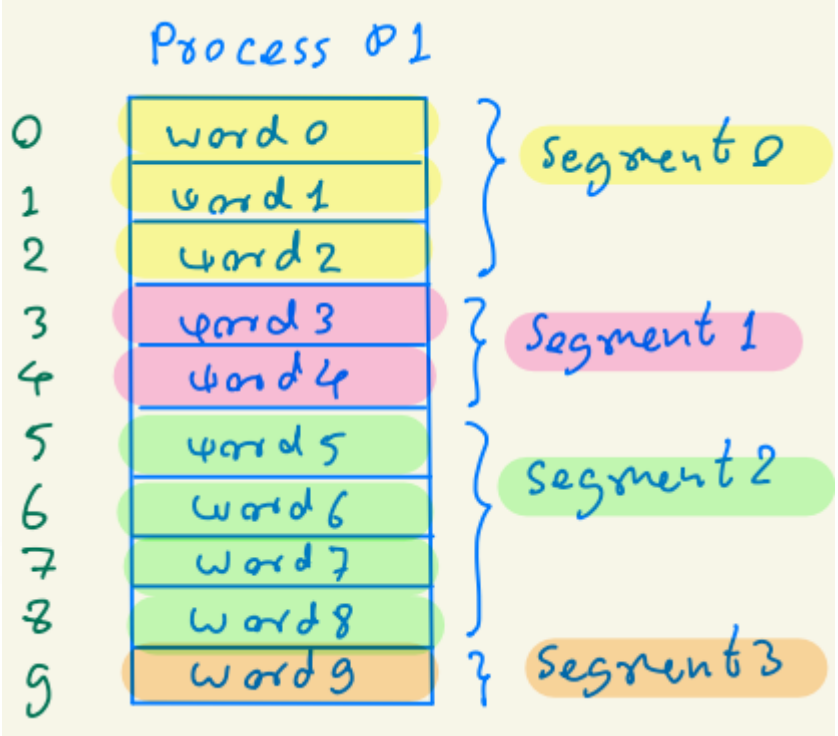
Segmentation

Segmentation is a memory management technique where a process is divided into multiple segments based on its logical structure. Each segment represents a logical unit such as a code module, data structure, stack, or heap.

- We will learn:
  a) Strategy to select free spaces in the main memory to store processes.
  b) Logical-physical address translation (mapping).

Key Characteristics of Segmentation

1. **A process is divided into multiple segments**
   - Unlike paging (where memory is divided into fixed-size pages), segmentation divides a process into variable-sized segments that correspond to logical divisions of the program.
   - Examples of segments include code segment, data segment, stack segment, and heap segment.
2. **Each segment has a different size**
   - Segments are not uniform in size; they are allocated memory based on their actual requirements.
   - For example, a code segment might require 10KB, while a stack segment may only need 2KB.

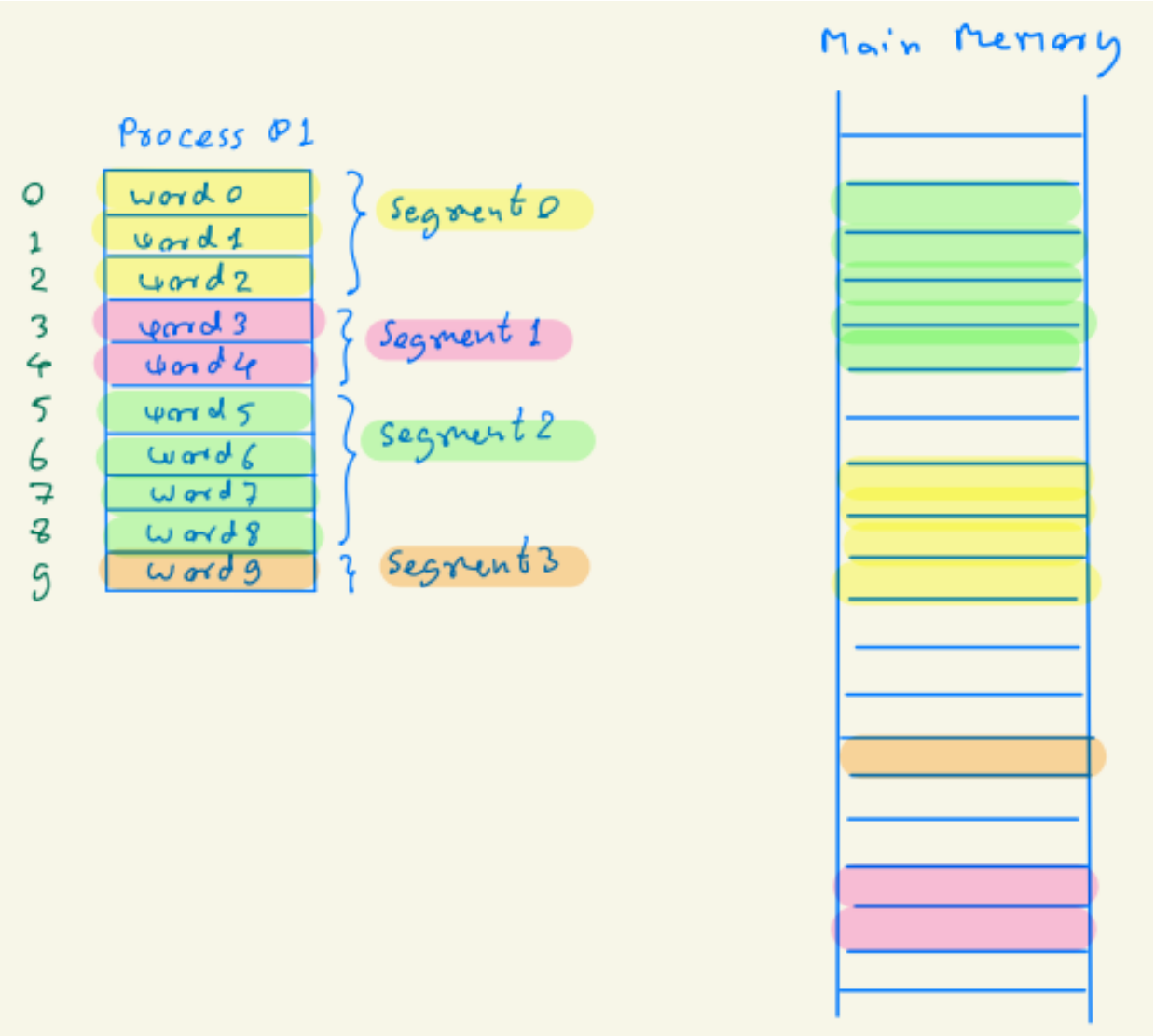


Key Characteristics of Segmentation

3) **Segments are stored separately in main memory**
   - Segments do not have to be stored contiguously in memory. They can be scattered across different locations in physical memory, depending on available space.
   - The segment table is used to keep track of each segment's base address (starting location) and length (size).
4) **To store a segment of a process into the main memory, we find a free space in the main memory that is large enough and we put this segment in this area.**
5) **Segmentation can suffer from external fragmentation.**

Example: Segmentation

The following example shows how to store processes in the main memory by using the segmentation.



Logical Address in Segmentation

1) Segmentation is a memory-management scheme that supports a programmer view of memory. As a result, a process is divided into segments.
2) Each segment can have a different size.
3) Each segment has its own logical address specified by two quantities (a tuple):

< s, d >

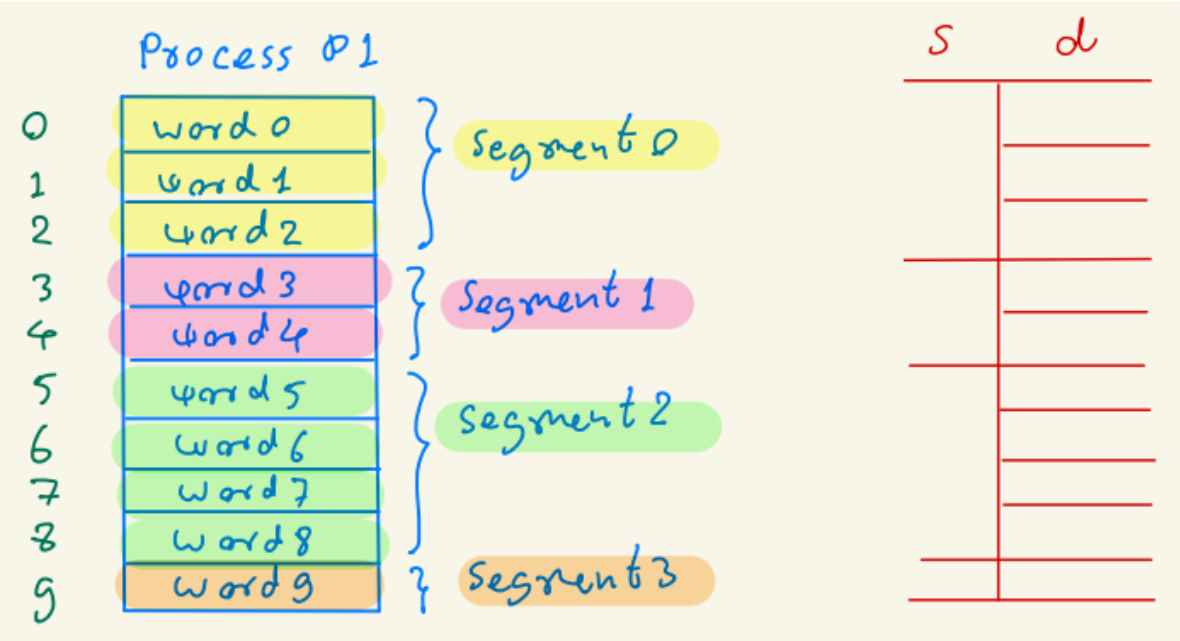
- The segment number (s): Identifies the segment within the process. It is an integer starting from 0.
- An offset in this segment (d): Specifies the location of a word within the segment. It is an integer starting from 0. The maximum offset is determined by the segment's size (limit).

Example: Logical address (segmentation)

The following example shows an example of logical addresses of a process according to segmentation.

a) What is the logical address <s,d> of word6?

b) Which word is stored at the logical address <1,0>?



Logical-Physical Mapping: Segment Table

1) The segmentation identifies the locations in the main memory to store segments of a process by using the segment table.
2) Each process has its own segment table.
3) Each entry in the segment table has a segment limit and a segment base.
   - The segment limit specifies the length/size of the segment (i.e., the number of words in that segment). It implies that the offset d is between 0 and limit-1.
   - The segment base contains the starting physical address where the segment resides in memory.

	limit	base
0	1000	1400
1	400	6300
2	400	4300
3	1100	3200
4	1000	4700

segment table

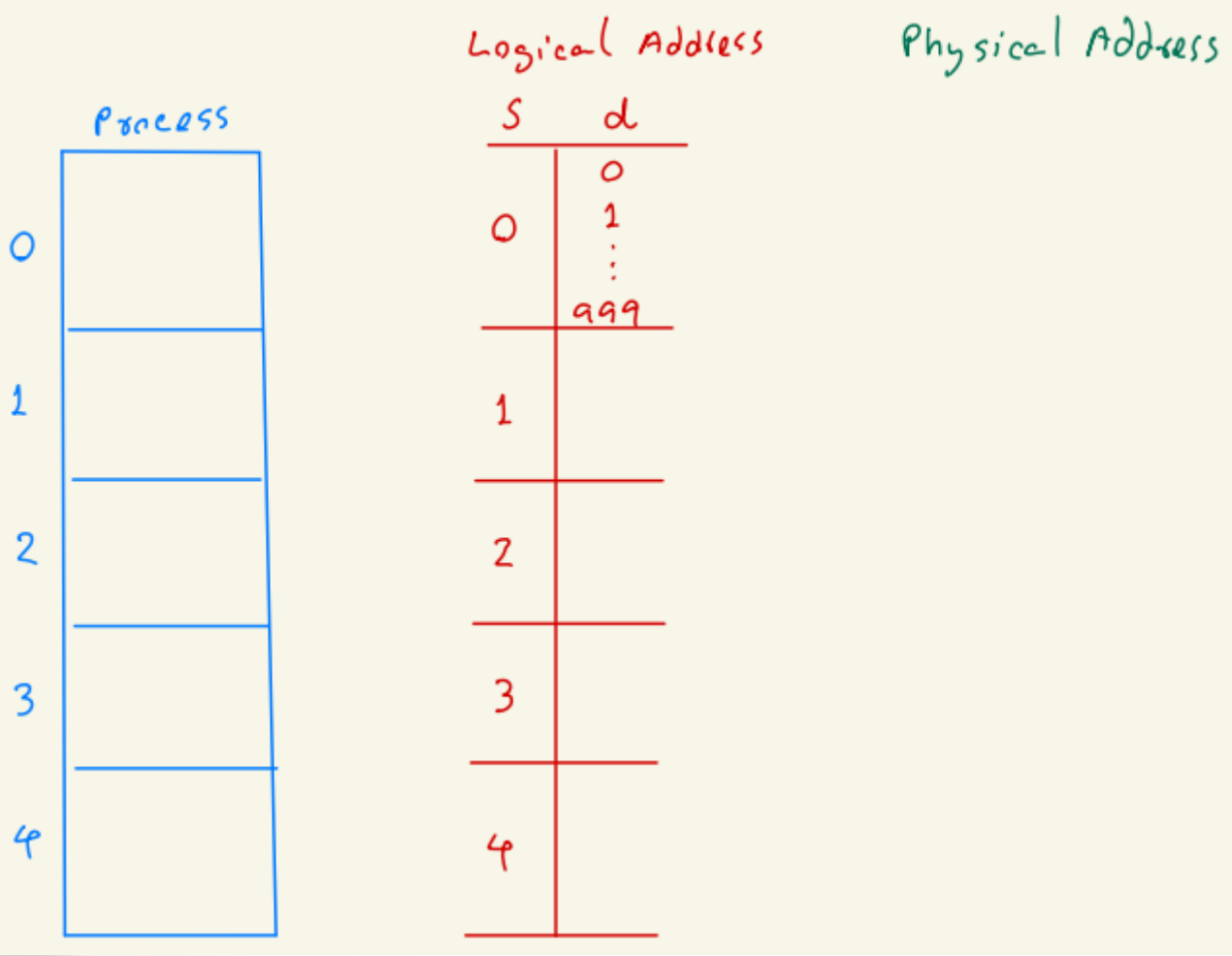
Example: Segment table and Logical Addresses

Consider the following segment table of a process.

- Draw a logical diagram of this process to show the logical addresses.

	limit	base
0	1000	1400
1	400	6300
2	400	4300
3	1100	3200
4	1000	4700

segment table

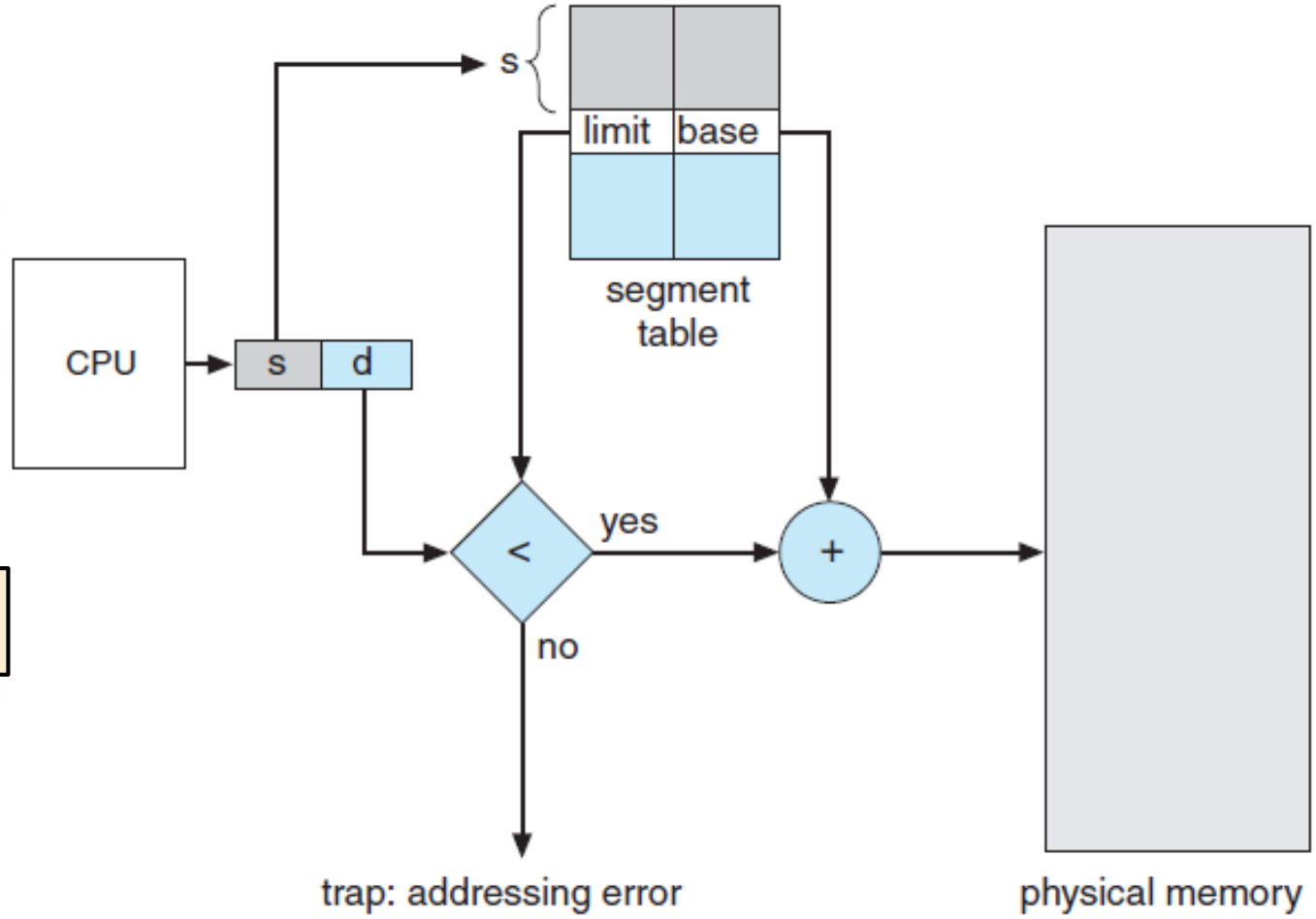


Logical-Physical Mapping: Physical Address

The physical address corresponding to the logical address <s,d> can be computed as follows.

1) Check whether the offset (d) is valid or not:
   - If d < limit, go to Step 2.
   - If d > or = limit, error and invalid.
2) The physical address is

Physical Address = base + d



Example: Physical Addresses (Example 1)

Consider the segmentation method to store a process in the main memory. The following segment table is given. Answer the following questions.

1. Find the physical address of the logical address <2, 53>.
2. Find the physical address of the logical address <3, 852>
3. Find the physical address of the logical address <1, 400>.

	limit	base
0	1000	1400
1	400	6300
2	400	4300
3	1100	3200
4	1000	4700

segment table

Example: Big Picture of Segmentation Method

Consider the segmentation method to store a process in the main memory. The following segment table is given. Draw a diagram to show locations of these 5 segments in the main memory.

	limit	base
0	1000	1400
1	400	6300
2	400	4300
3	1100	3200
4	1000	4700

segment table

Ex Physical address of logical addresses

a) < 1, 150 >

b) < 4, 1050 >

Process P0

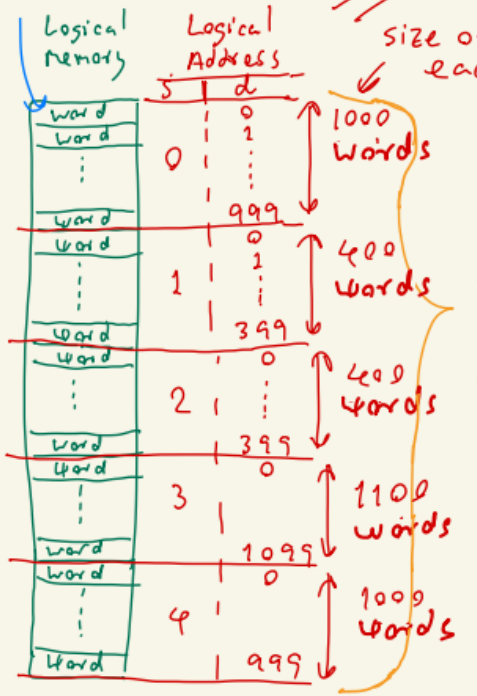
Segment Table

S	limit	base
0	1000	1400
1	400	6300
2	400	4300
3	1100	3200
4	1000	4700

starting physical address

show here as a decimal number → Physical Address

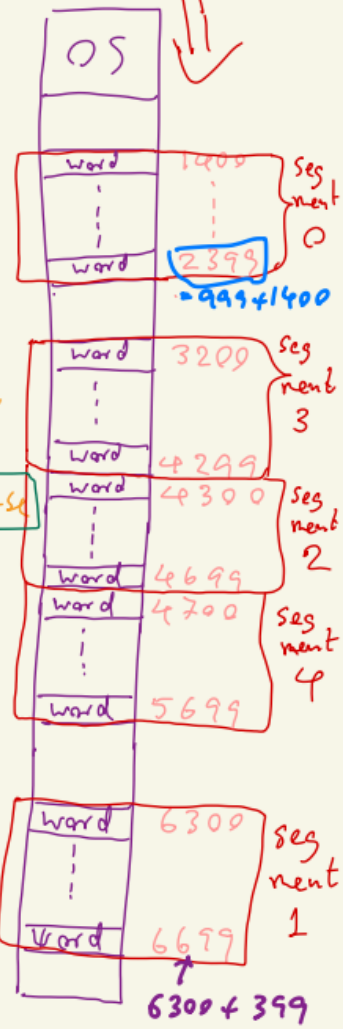
size of each segment



Mapping:

Put the word from a logical address to the corresponding physical address.

- if d < limit
  - ↳ Phy Addr = d + base
- if d ≥ limit
  - ↳ error



5. Method 3: Paging

Paging is another memory-management scheme that permits the physical address space of a process to be non-contiguous.

By using paging, we can avoid the external fragmentation problem.

Paging

Paging is a memory management technique that allows the physical address space of a process to be non-contiguous, eliminating external fragmentation (*but internal fragmentation exits*) and simplifying memory allocation. Unlike segmentation, which divides memory into variable-sized segments, paging divides memory into fixed-size blocks, making it more efficient for managing memory.

- We will learn:
  a) Strategy to select free spaces in the main memory to store processes.
  b) Logical-physical address translation (mapping).

Overview of Key Characteristics of Paging

1) **Processes are divided into fixed-size pages**
   - A process is split into equal-sized pages.
   - Each page contains the same number of words, ensuring uniform size.
   - Pages do not need to be stored contiguously in physical memory.
2) **Physical memory is divided into frames**
   - Main memory is partitioned into fixed-size frames, each of which can hold exactly one page.
   - The frame size is equal to the page size, ensuring compatibility.

Key Characteristics of Paging

3) **Mapping Logical to Physical Address**
   - A page table is used to map a process's logical pages to available physical frames in memory.
   - Each entry in the page table contains:
     - Page number (index in the table).
     - Frame number (location in physical memory).
4) **Paging can suffer from internal fragmentation**.

Logical Addresses in Paging (of a Process)

1) The logical addresses of a process is divided into fixed-size blocks called pages (p).
2) Each page consists of the same number of words (= page size).
3) Each word in a page can be identified by an offset number (d).
4) As a result, a logical address of a word in a process can be represented by

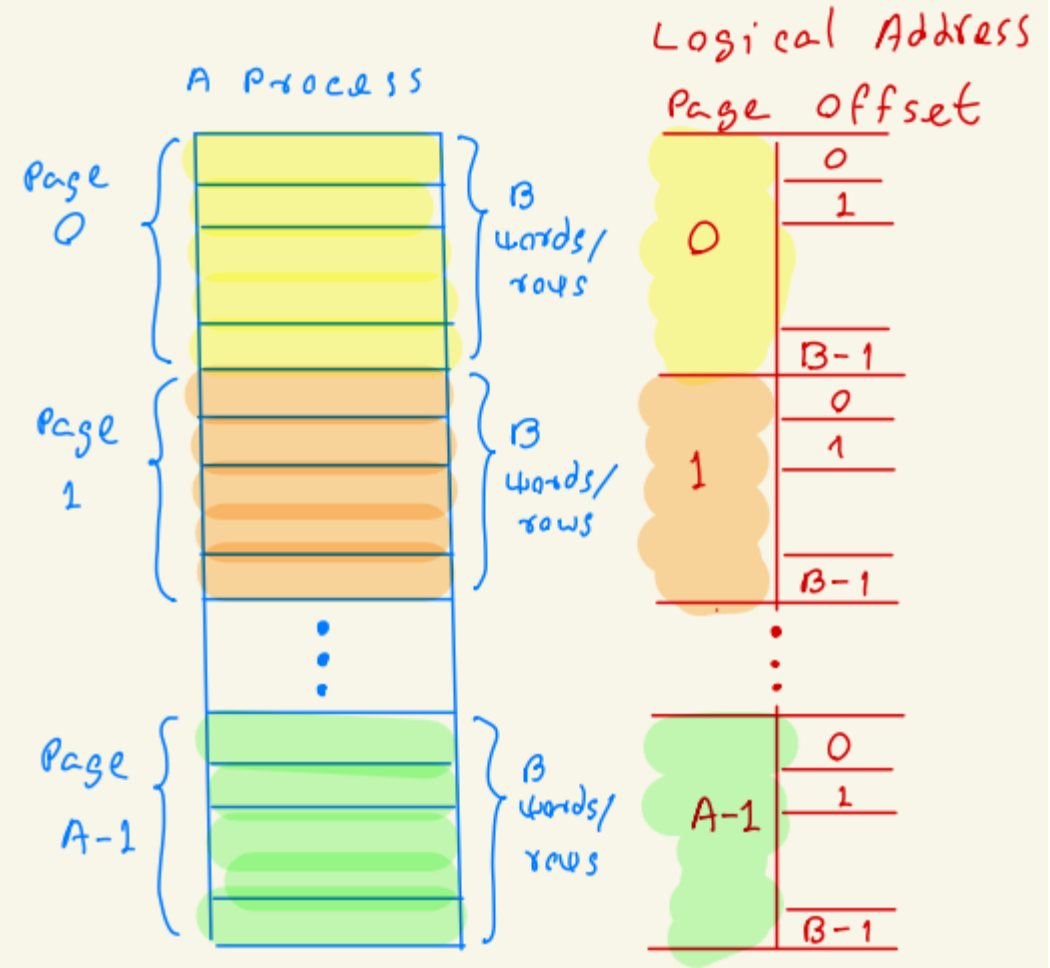
<p, d>

- The page number (p): Identifies the page within the process. It is an integer starting from 0.
- An offset in this page (d): Specifies the location of a word within the page. It is an integer starting from 0. The maximum offset is determined by the page size.

Illustration: Logical Addresses in Paging

This diagram shows an example of the logical address of a process based on the paging technique (where each page consists of B words).

- Assume that this process consist of A pages and each page consists of B words (B offsets).
- We have the following logical addresses.



Example: Logical Addresses of a Process

Consider a paging method where each page consists of 4 words. A process named P1 consists of 8 words. The following diagram shows an example of logical addresses of the process.

1) What is the logical address <p,d> of word6?

2) Which word is at the logical address <0,2>?

logical address	P1
0	word 0
1	word 1
2	word 2
3	word 3
4	word 4
5	word 5
6	word 6
7	word 7

logical address p	d
0	0
	1
	2
	3
1	0
	1
	2
	3

Physical Addresses in Paging (in Main Memory)

1) The physical addresses of a process is divided into fixed-size blocks called frames (f).
2) Each frame consists of the same number of words (frame size). (The frame size is equal to the page size.)
3) Each word in a frame can be identified by an offset number (d).
4) As a result, a physical address of a word in the main memory can be represented by

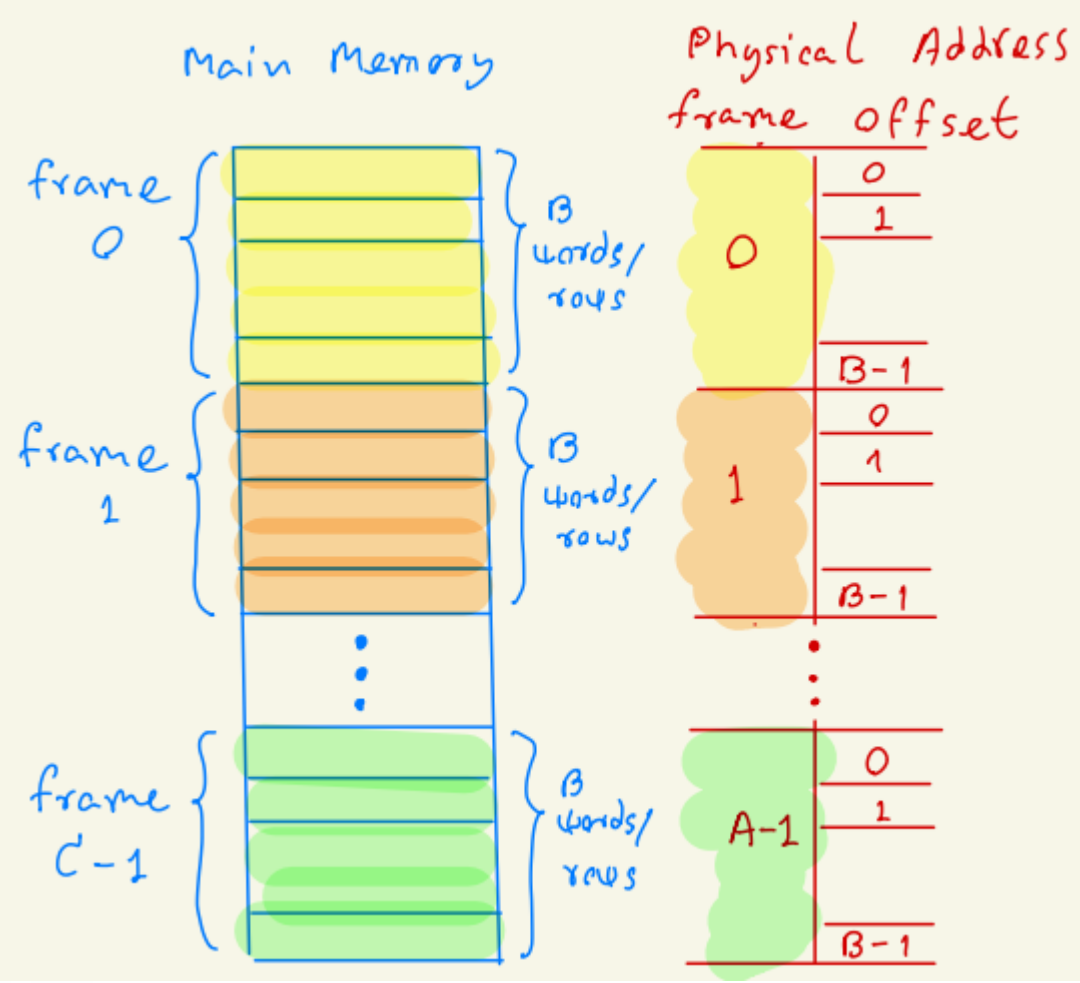
<f, d>

- The frame number (f): Identifies the frame within the main memory. It is an integer starting from 0.
- An offset in this frame (d): Specifies the location of a word within the frame. It is an integer starting from 0. The maximum offset is determined by the frame size. The offset d in the page is the same with the offset d in the corresponding frame.

Illustration: Physical Addresses in Paging

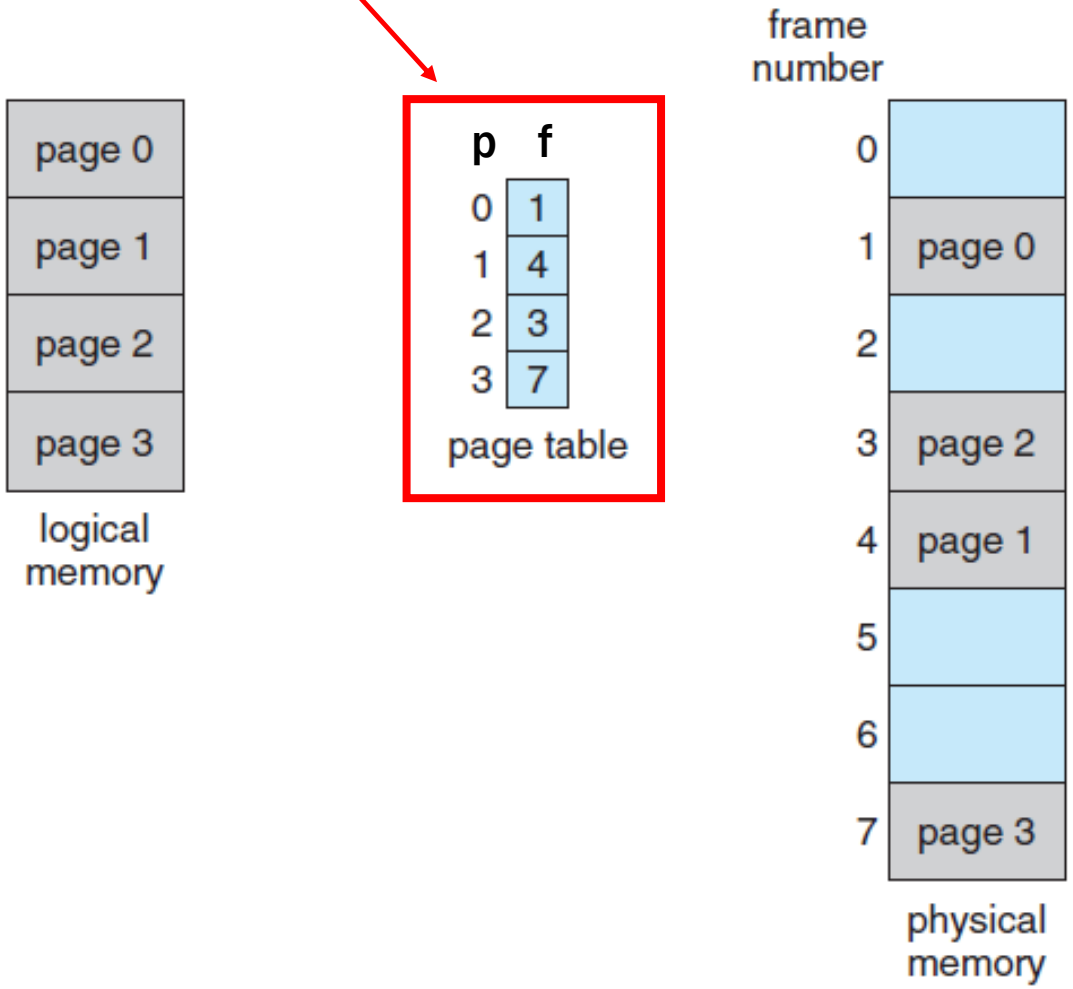
This diagram shows an example of the physical address of the main memory based on the paging technique (where each page consists of B words).

- Assume that this main memory consist of C frames and each frame consists of B words (B offsets).
- We have the following physical addresses.



Page Table (Mapping Page to Frame)

The page table consists of two columns: page number (p) and frame number (f). It shows mapping between the page numbers (p) and the frame numbers (f).



For example:

- Page 0 will be put into Frame 1.
- Page 1 will be put into Frame 4.
- Page 2 will be put into Frame 3.
- Page 3 will be put into Frame 7.

Logical-Physical Mapping

By using the page table, we can map between logical address <p, d> and the physical address <f, d>.

Logical Address <p, d> ⟷ Page Table p ←→ f ⟷ Physical Address <f, d>

Example: Logical-Physical Mapping

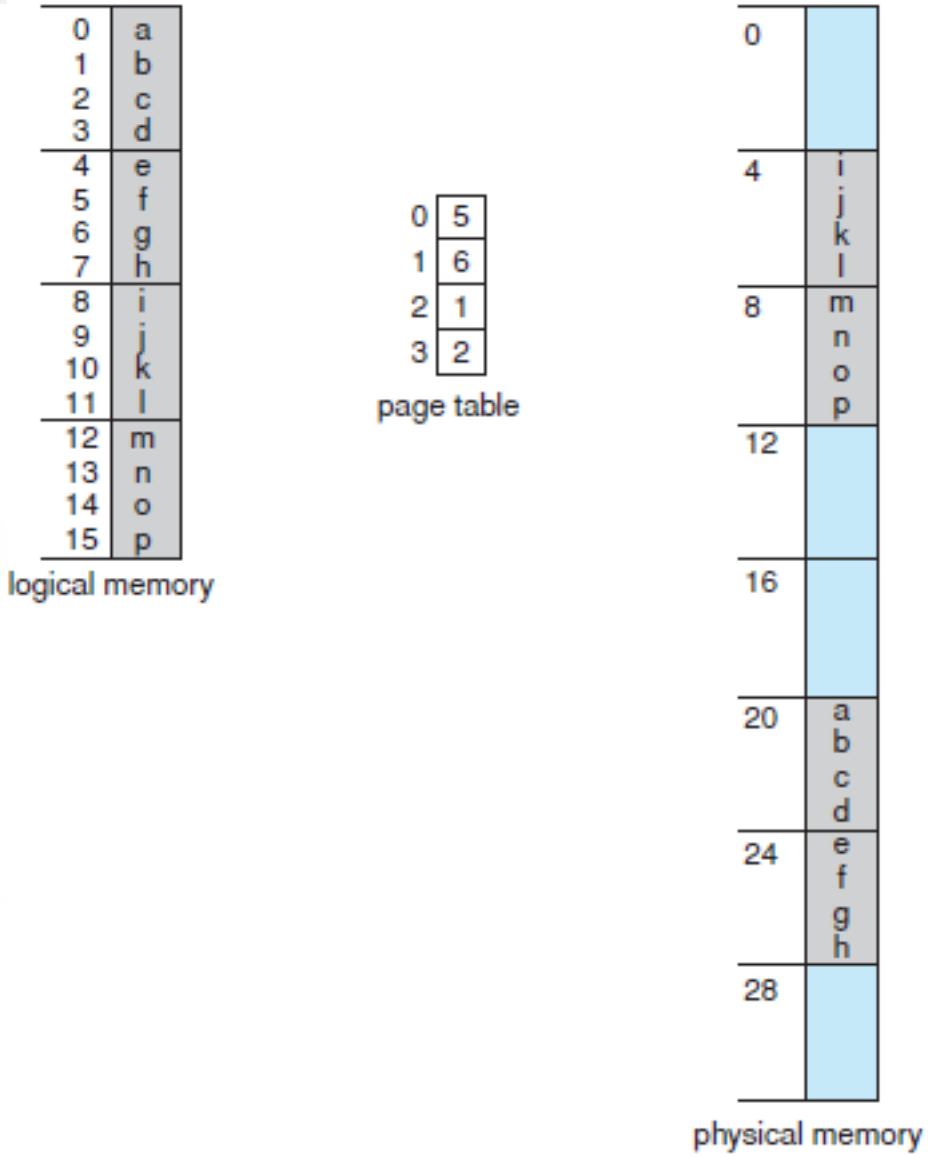
Consider a process whose logical space is divided into 4 pages. Each page consists of 4 bytes (where each word is equal to 1 byte). The process are mapped/stored into the memory which is divided into 8 frames. Each frame consists of 4 bytes. The page table is given as shown below. Answer the following questions.

1) How many words are in a frame?

2) What is the logical address <p, d> of word "i"?

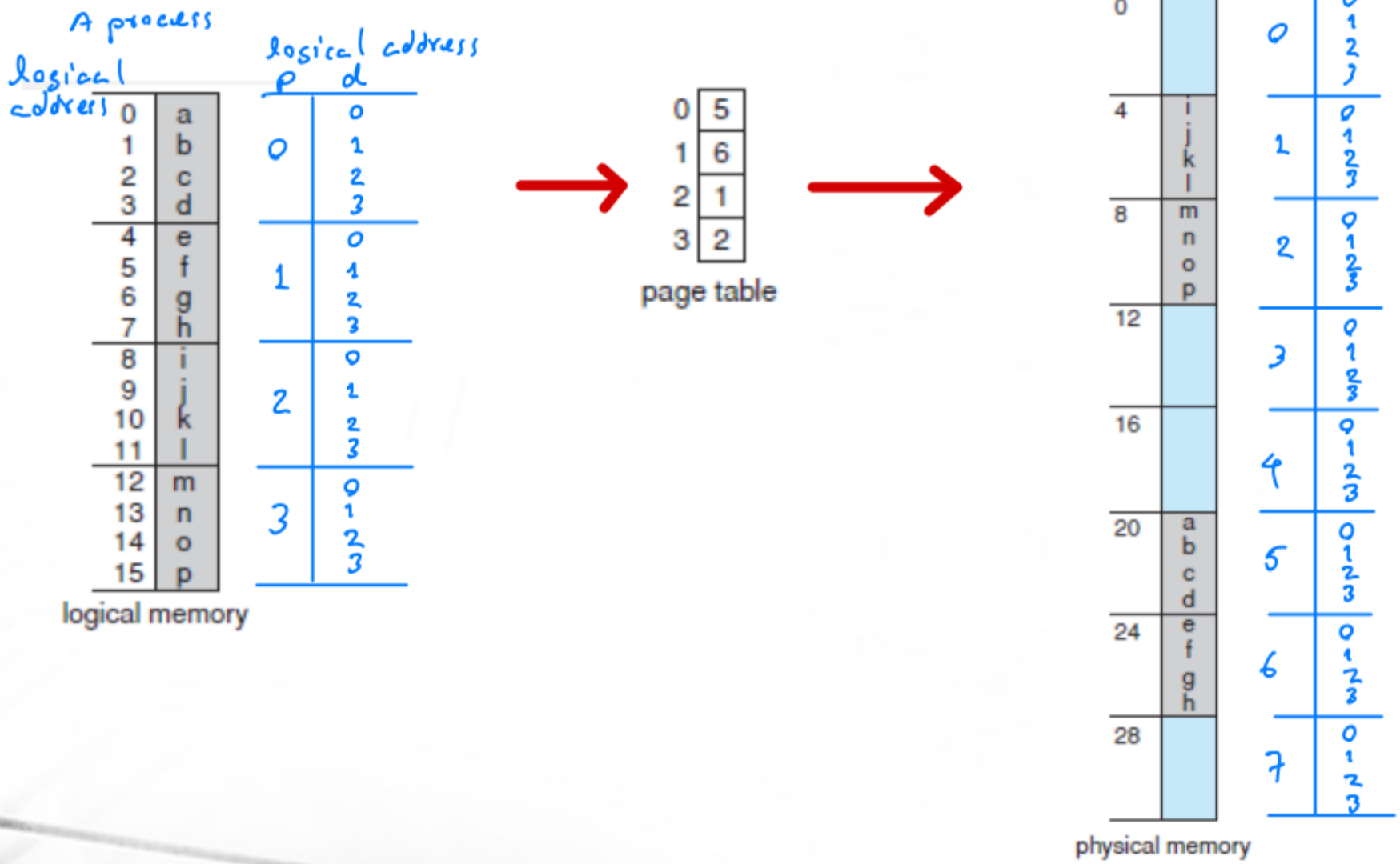
3) What is the physical address <f, d> of word "i"?

4) What is the physical address <f, d> of word "d"?



Example: Big Picture

The following diagram concludes the paging method.



6. Address Conversion in Paging

- Logical Address Conversion
- Physical Address Conversion

Logical Address Conversion

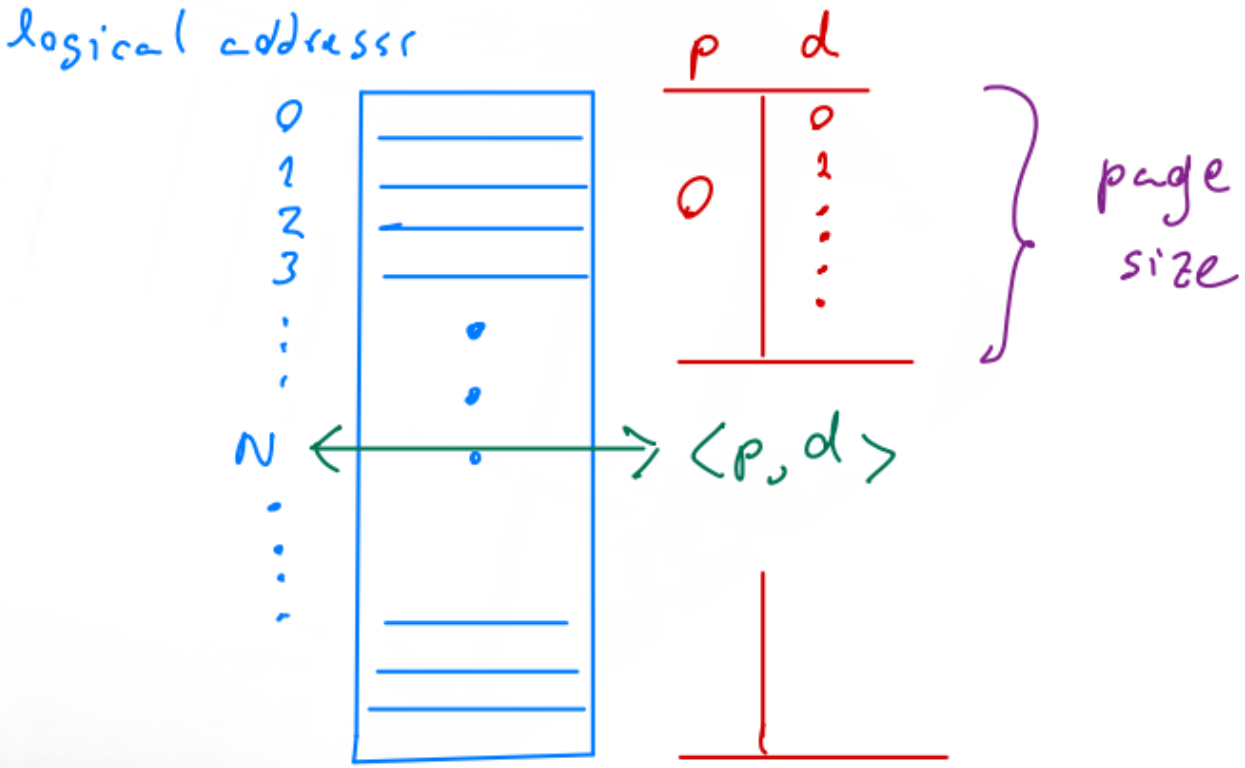
A logical address can be represented by

1) A decimal number
2) A tuple <p, d>
3) A binary number

Logical Address Conversion between Decimal and <p, d>

We can convert between decimal representation (N) and <p, d> representation as follows. Note that the page size is in the unit of words.

$$\frac{N}{\text{Page Size}} = p \quad \frac{d}{\text{Page Size}}$$



Example: Logical Address Conversion between Decimal and <p, d>

Consider a paging method whose page size is equal to 4 words.

1) Find the logical address <p, d> corresponding to the logical address 5 (decimal representation).

2) Find the logical address in the decimal representation of the logical address <0, 2>.

Example: Logical Address Conversion between Decimal and <p, d>

Assuming a 2-KB (2048 bytes) page size, what are the page numbers and offsets for the following address references (provided as decimal numbers): 15000? Assume that 1 word is equal to 1 byte.

Logical Address Conversion between <p, d> and Binary Number

- A *logical address in paging* can be represented by an *m*-bit binary number:

<p, d> ⟷

page number	page offset
p	d
$m - n$	n

  - A page number (p) is represented by the first m-n bits.
  - A page offset (d) is represented by the last n bits.
- From the above logical address, the logical space consists of:
  - the number of pages is equal to 2^{m-n}. (Unit: pages)
  - the page size is equal to 2^{n}. (Unit: words)

Example: Logical Address Conversion between <p, d> and Binary Number

Consider a paging method whose logical address can be represented by a 7-bit binary number and the offset number (d) can be represented by a 4-bit binary number.

1) What is the page size?

2) What is the maximum number of pages that we can have?

Example: Logical Address Conversion between <p, d> and Binary Number

1) If m=5 and n=3, what is the <p, d> of the logical address 10110 (binary number)?

2) If m=6 and n=2, what is the binary number to represent the logical address <5, 1>?

Physical Address Conversion

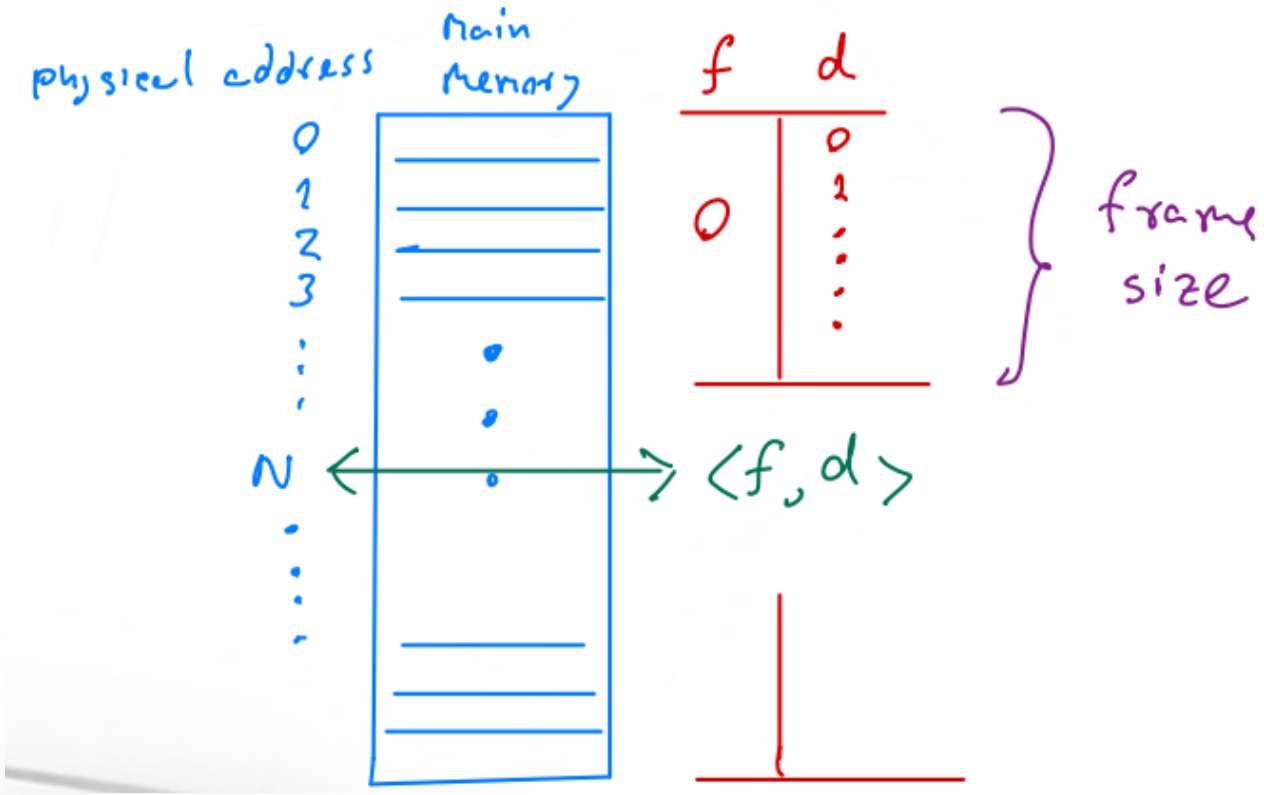
A physical address can be represented by

1) A decimal number
2) A tuple <f, d>
3) A binary number

Physical Address Conversion between Decimal and <f, d>

We can convert between decimal representation (N) and <f, d> representation as follows.

$$\frac{N}{\text{Frame Size}} = f \frac{d}{\text{Frame Size}}$$



Example: Physical Address Conversion between Decimal and <f, d>

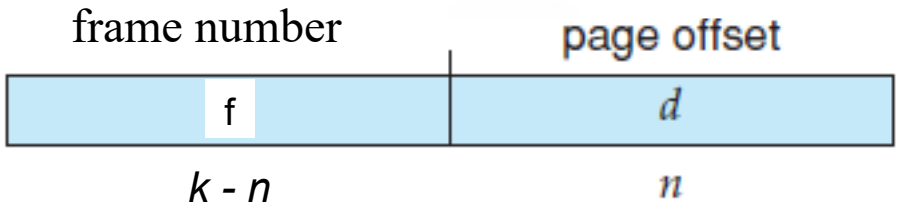
Consider a paging method whose page size is equal to 32 words and page table is shown below. Find the physical address in the decimal representation of the logical address <2, 20>.

0	1
1	4
2	3
3	7

page table

Physical Address Conversion between <f, d> and Binary Number

- A *physical address in paging* can be represented by an *k*-bit binary number:



  - A frame number (f) is represented by the first k-n bits.
  - A page offset (d) is represented by the last n bits.
- From the above physical address, the physical space consists of:
  - the number of frames is equal to 2^{k-n} frames.
  - the frame size is equal to 2^{n} words.

Example: Physical Address Conversion between <f, d> and Binary Number

Consider a paging method whose physical address can be represented by a 9-bit binary number and the offset number (d) can be represented by a 5-bit binary number.

1) What is the frame size?

2) What is the maximum number of frames in the main memory that we can have?

Example: Physical Address Conversion between <f, d> and Binary Number

1) If k=6 and n=2, what is the <f, d> of the logical address 10110 (binary number)?

2) If k=7 and n=3, what is the binary number to represent the physical address <7, 2>?

Example

Consider a logical address space of 64 pages of 1024 words each, mapped onto a physical memory of 512 frames.

a) What is the minimum number of bits to represent physical addresses?

b) What is the minimum number of bits to represent physical addresses?

7. Internal Fragmentation in Paging

We will compute internal fragmentation.

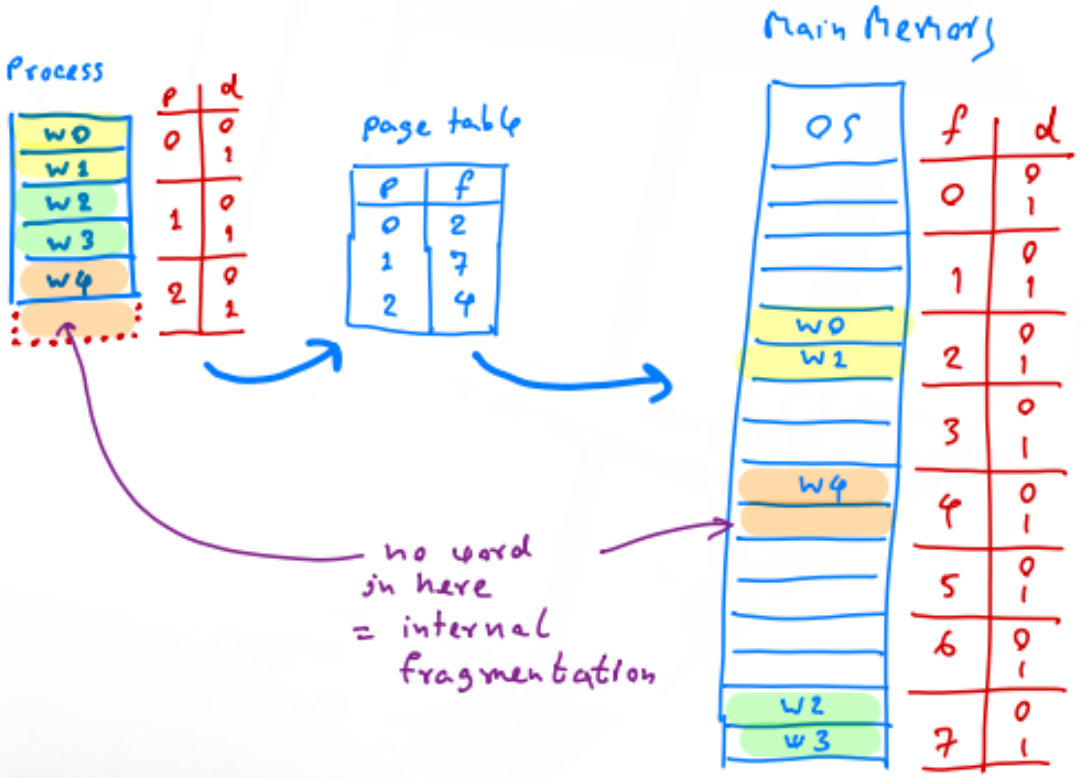
Internal Fragmentation

Internal fragmentation refers to a free space (not usable) in a frame (physical address) and happens in the paging techniques.

- Equivalently, the free space area (unused) in the last page of a process.
- This happens because the size of a process (= the number of words) is not divisible by the page size.

Example 1: Internal Fragmentation

A process consists of 5 words. If we use a paging method with page size equal to 2 words. The process will be divided into 3 pages where the last page contains one word and a blank space. Corresponding, when we store this process into the main memory, the frame who store the last page will also have a blank space. This blank space is referred as an internal fragmentation.



Steps to Compute the Internal Fragmentation

The steps are shown below.

① Find the number of pages to store this process.

$$\#\text{pages } (V) = \left\lceil \frac{\text{size of process (unit: words)}}{\text{page size (unit: words)}} \right\rceil$$

② Find the maximum number of words covered by this V,

$$\text{Num}_{max} = V \times \text{Page Size (unit: words)} \quad \text{(unit: words)}$$

③ Internal fragmentation

$$= \text{Num}_{max} \text{ (unit: words)} - \text{size of the process (unit: words)}$$

Remark: Instead of the unit of words (as shown above), the unit of bits or Bytes can be used but all variables above must apply the same unit (such as words as example above).

Example 2: Internal Fragmentation

Consider a process of 72766 bytes and a page size is 2048 bytes. Assume that the word size is equal to 8 bits. How much is the internal fragmentation?

8. Page Table and Frame Table

- Page table
- Frame table

Page Table in Paging

- A **page table** is a **data structure** used in **paging systems** to translate **logical addresses** (generated by a process) into **physical addresses** (used by the memory hardware). It plays a critical role in **memory management** by mapping **pages** in a process's logical address space to **frames** in physical memory.
- Typically, a page table consists of the following entries.

Field	Description
Page Number	Index of the page in the logical address space.
Frame Number	Corresponding frame in **physical memory** where the page is stored.
Valid/Invalid Bit	Indicates if the page is **in memory (1)** or **not (0, meaning it's on disk and causes a page fault).**
Dirty Bit	Set if the page has been **modified** (helps decide if the page needs to be written back to disk).
Access/Protection Bits	Defines read/write/execute permissions for memory protection.

Valid and Invalid Bits in a Page Table

- Consider an m-bit logical space (consisting of logical addresses 0 to 2^m). If a process has a size of 2^i (which is less than 2^m), there might be some pages unused.
- In the page table, there is one more column to identify whether each page is *valid* (used) or *invalid* (unused).
  a) We use v or "1" to identify a valid page. Specifically, v identifies that this page is stored in the main memory.
  b) We use i or "0" to identify an invalid page. There are 2 possible meanings.
     i. We do not have this page.
     ii. On the other hand, if we have this page, now, this page is not stored in the main memory (i.e., it is currently in the hard disk). In the frame number we might put "-1" and we will have information on where it is in the hard disk.
- The unused/invalid pages will not be mapped to frames.

Dirty Bits in a Page Table

- If the dirty bit is set (1) → The page has been modified and must be written back (saved) to disk before being replaced (or removed/erased from the main memory).
- If the dirty bit is not set (0) → The page has not been modified, so it can be replaced without saving it.

Example: Page Table

Page Number	Frame Number	Valid Bit	Protection	Dirty Bit
0	2	1	Read-Write	0
1	0	1	Read-Write	1
2	3	1	Read-Only	0
3	- (Invalid)	0	-	-

From the frame table above:

- Page 0 is stored in Frame 2.
- Page 1 is stored in Frame 0 with a dirty bit set (needs to be written back before replacement).
  - If the dirty bit is set (1), the page has been modified and must be written (= saved) back to disk before being replaced (= removed/erased).
  - If the dirty bit is not set (0), the page has not been modified, so it can be replaced without saving it.
- Page 3 is invalid (i.e., we might not have it or it is now stored in the hard disk).

Example: Page Table

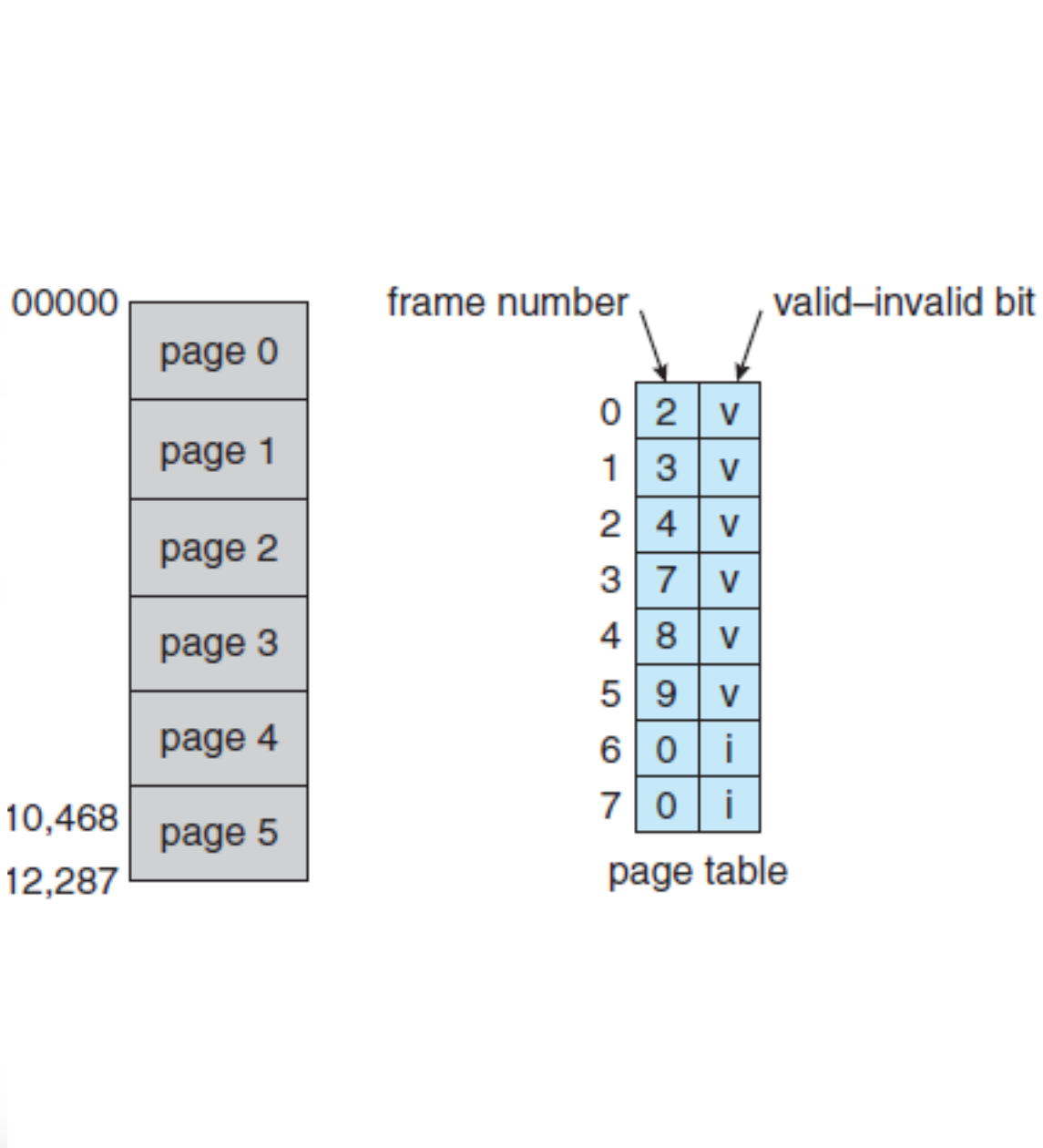
Page Number	Frame Number	Valid Bit	Protection	Dirty Bit
0	2	1	Read-Write	0
1	5	1	Read-Write	0
2	1	1	Read-Write	1
3	-	0	Read-Write	0
4	-	0	Read-Write	0
5	3	1	Read-Only	0
6	-	0	Read-Write	0
7	-	0	Read-Write	0

From the frame table above:

1) Since the valid bit is 0, does this process have Page 3?
2) Since the valid bit is 0, does this process have Page 6?

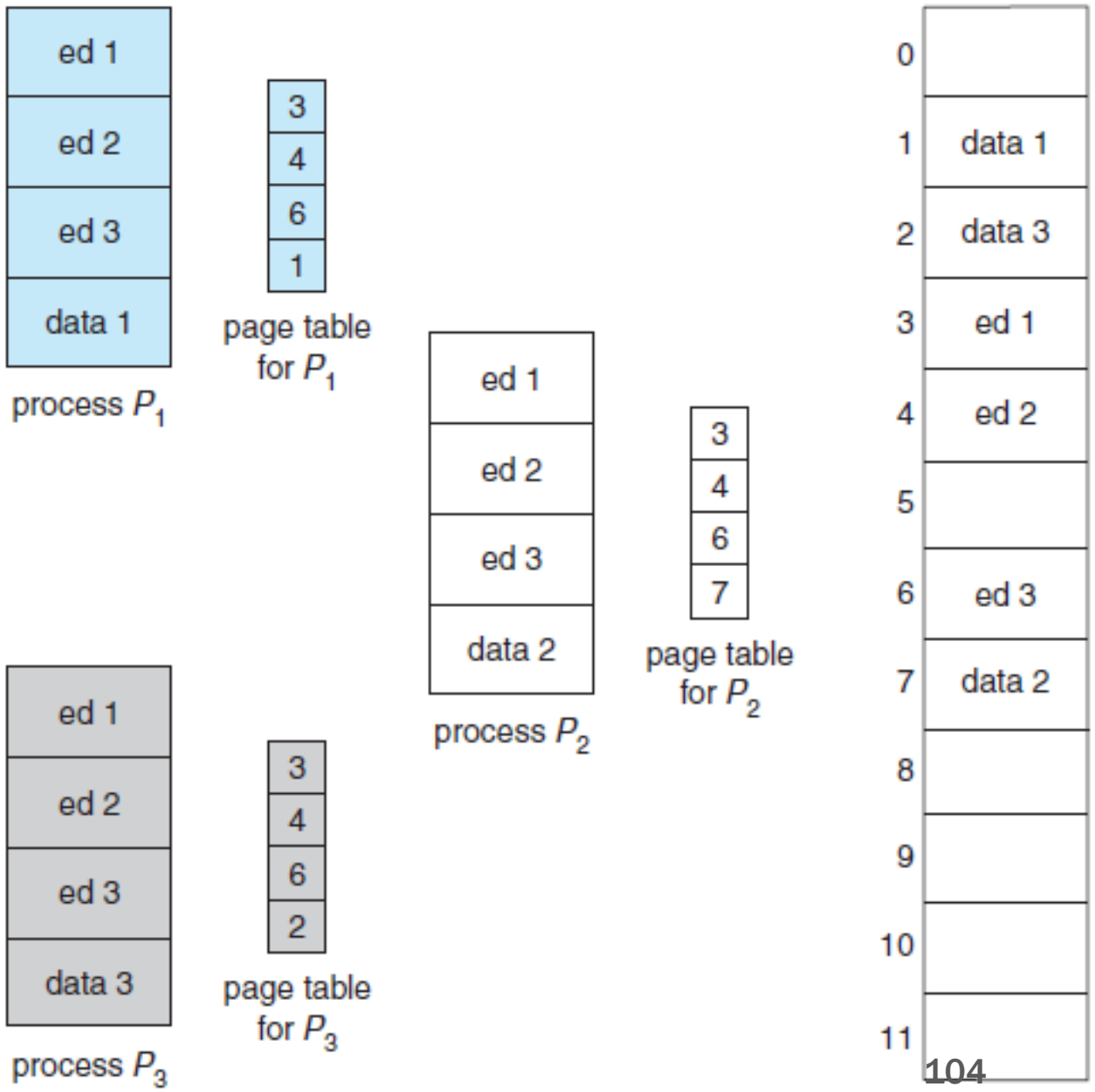
Example: Valid and Invalid Pages

Suppose, for example, that in a system with a 14-bit address space (0 to 16383), we have a program that should use only addresses 0 to 10468. Given a page size of 2 KB, we have the situation shown in the figure. Addresses in pages 0, 1, 2, 3, 4, and 5 are mapped normally through the page table.



Shared Pages

- Many processes might have similar operations and segment of codes.
- As a result, some pages of these processes might be identical.
- When mapping to frames (physical address), these processes might share the same frames.



Questions

1) Is it possible that the process P1 has a word at the logical address 100 and the process P2 is also has a word at the logical address 100?

   Ans:

2) Is it possible that the process P1 has a word at the physical address 100 and the process P2 is also has a word at the physical address 100?

   Ans:

Frame Table

- A frame table is a data structure used in paging systems to manage physical memory allocation. It keeps track of which frames in main memory are currently allocated and which are available for use.
- Typically, a frame table consists of the following entries.

Field	Description
Frame Number	Unique identifier of the memory frame.
Process ID (PID)	The ID of the process using this frame.
Page Number	The corresponding **page** stored in this frame.
Status Bit	Indicates if the frame is **free (0)** or **allocated (1)**.
Dirty Bit	Set if the page has been modified (used for page replacement).
Protection Bits	Defines read/write/execute permissions.

Example: Frame Table

Frame #	Process ID (PID)	Page #	Protection	Dirty Bit
0	2	1	Read-Write	1
1	1	4	Read-Only	0
2	3	2	Read-Write	1
3	- (Free)	-	-	-

From the frame table above:

- Frame 0 belongs to process 2 and holds page 1 with a dirty bit set (needs to be written back before replacement).
  - If the dirty bit is set (1), the page has been modified and must be written (= saved) back to disk before being replaced (= removed/erased).
  - If the dirty bit is not set (0), the page has not been modified, so it can be replaced without saving it.
- Frame 1 belongs to process 1 and holds page 4.
- Frame 3 is free and available for allocation.

Reading Assignment

- Section 8.1.1, 8.2, 8.3, 8.4, 8.5.1, 8.5.3, 8.5.4

Glossary (by Google NotebookLM)

- **Main Memory (RAM):** The primary storage area that holds active processes and data, allowing the CPU to access them directly.
- **Hard Disk (Secondary Storage/Backing Store):** A non-volatile storage device used to store programs and data that are not currently being actively used by the CPU.
- **Process:** An instance of a computer program that is being executed.
- **Logical Address (Virtual Address):** An address generated by the CPU when a program accesses memory; it is process-specific.
- **Physical Address:** The actual address of a location in the main memory (RAM).
- **Memory Management Unit (MMU):** A hardware component responsible for translating logical addresses to physical addresses.
- **Contiguous Memory Allocation:** A memory management technique where each process is allocated a single, continuous block of memory.
- **Fixed-Size Partition:** A contiguous memory allocation method where the main memory is divided into partitions of equal or unequal, but predetermined, sizes.

Glossary (by Google NotebookLM)

- **Variable Partition (Dynamic Partition):** A contiguous memory allocation method where memory is allocated dynamically to processes based on their size, creating holes of free memory.
- **Internal Fragmentation:** Wasted space within an allocated memory block (partition or frame) because the process using it is smaller than the allocated size.
- **External Fragmentation:** A situation where there is enough total free memory in the system to satisfy a request, but it is scattered in non-contiguous blocks.
- **Swapping:** A memory management technique where an entire process is moved between main memory and secondary storage (swap space).
- **Segmentation:** A memory management technique where a process is divided into logical units called segments, which can be of varying sizes and stored non-contiguously in memory.
- **Segment Table:** A data structure used in segmentation to store the base address and limit (size) of each segment of a process.
- **Paging:** A memory management technique where both the process's logical address space and the physical memory are divided into fixed-size blocks called pages and frames, respectively.

Glossary (by Google NotebookLM)

- **Page Table:** A data structure used in paging to map logical page numbers to physical frame numbers.
- **Frame:** A fixed-size block of physical memory that can hold one page.
- **Page:** A fixed-size block of the logical address space of a process.
- **Page Offset:** The displacement within a page or frame.
- **Frame Number:** The identifier of a frame in physical memory.
- **Page Number:** The identifier of a page in the logical address space of a process.
- **Valid Bit:** A bit in a page table entry that indicates whether the corresponding page is currently residing in a valid physical frame in memory.
- **Invalid Bit:** A bit in a page table entry that indicates that the corresponding page is not currently in main memory.
- **Dirty Bit:** A bit in a page table entry that indicates whether the page has been modified since it was loaded into memory.
- **Frame Table:** A data structure that keeps track of the status of each physical frame in memory (e.g., allocated or free, and if allocated, to which process and page).